

memory and collection

09/03/2026 12:10:23

i will give you the java question you have to answer it according to 5 yr of exp in...
Absolutely. 👍 Give me any Java interview question, and I'll answer it at the level...
Why is Java pass-by-value even when we pass an object?

This is a very common 5-year Java interview question. The key is to clearly...
just give me the answer like a note not that much explanation

Why is Java pass-by-value even when we pass an object? Interview Answer: Java...
What exactly happens when new Employee() is executed? How does Garbage...

1. What exactly happens when new Employee() is executed? Interview Answer:...
What is the Object class in Java? What is equals()? What is the contract of...

Java Object, equals(), hashCode() & Immutability — Interview Notes 1. What is t...
How do you create an immutable class? Why is String immutable? What is the...

Immutability, String & HashSet — Interview Notes 1. How do you create an...

What happens when "Java" and new String("Java") are added to a HashSet? Wha...
1. What happens when "Java" and new String("Java") are added to a HashSet?...

How does HashMap work internally? What happens when map.put(key, value) is...
HashMap Internals — 5-Year Interview Notes 1. How does HashMap work...

What changed in Java 8 HashMap? Why does HashMap convert a bucket into a...
HashMap Advanced — 5-Year Interview Notes 1. What changed in Java 8...

What happens if two keys have the same hashCode? What happens if hashCode...
HashMap Edge Cases — 5-Year Interview Notes 1. What happens if two keys hav...

HashMap vs Hashtable HashMap vs ConcurrentHashMap HashMap vs...

HashMap & Map Variants — 5-Year Interview Notes 1. HashMap vs Hashtable...

What is the Java Collections Framework? What is the difference between...

Java Collections Framework — 5-Year Interview Notes 1. What is the Java...

How does ArrayList work internally? How is ArrayList different from an array? Wh...
ArrayList Internals — 5-Year Interview Notes 1. How does ArrayList work internall...

How does ArrayList store objects internally? Can ArrayList contain null? Can...

ArrayList — Advanced Interview Notes 1. How does ArrayList store objects...

How does LinkedList work internally? What data structure does LinkedList use?...

LinkedList — 5-Year Interview Notes 1. How does LinkedList work internally?...

If you frequently insert elements in the middle, would you always choose...

ArrayList vs LinkedList — 5-Year Interview Notes 1. If you frequently insert...

How does HashSet work internally? How does HashSet ensure uniqueness? Why...

HashSet — 5-Year Interview Notes 1. How does HashSet work internally? HashSe...

How does LinkedHashSet maintain insertion order? LinkedHashSet vs HashSet?...

LinkedHashSet — 5-Year Interview Notes 1. How does LinkedHashSet maintain...

How does TreeSet work internally? What data structure does TreeSet use? How...

TreeSet — 5-Year Interview Notes 1. How does TreeSet work internally? TreeSet is...

What changed in HashMap after Java 8? Why does HashMap use a Red-Black Tre...

HashMap — Java 8+ Interview Notes 1. What changed in HashMap after Java 8?...

How does LinkedHashMap work internally? HashMap vs LinkedHashMap? How...

LinkedHashMap — 5-Year Interview Notes 1. How does LinkedHashMap work...

What is Queue? What is the difference between Queue and Deque? What is...

Queue, Deque & PriorityQueue — 5-Year Interview Notes 1. What is Queue? Que...

How does Iterator work? What is ConcurrentModificationException? Why does thi...

Iterator & ConcurrentModificationException — 5-Year Interview Notes 1. How doe...

Why isn't HashMap thread-safe? What happens if multiple threads modify an...

Collections & Concurrency — 5-Year Interview Notes 1. Why isn't HashMap threa...

Why doesn't ConcurrentHashMap allow null keys/values? ConcurrentHashMap vs...

ConcurrentHashMap — 5-Year Interview Notes 1. Why doesn't ConcurrentHashM...

What is the Collections class? Difference between Collections.sort() and...

Collections Utility Class — 5-Year Interview Notes 1. What is the Collections class...

Difference between Arrays.asList() and List.of()? Why does List.of() not allow null?

Arrays.asList() vs List.of() — 5-Year Interview Notes 1. Arrays.asList() vs List.of()...

You need to store 10 million unique IDs and frequently check whether an ID exis...

Collection Selection — 5-Year Interview Notes 1. 10 million unique IDs + frequen...

You need an LRU cache. Which collection can help implement it? You have 90%...

Collection Selection — Scenario-Based Interview Notes 1. You need an LRU cach...

You need a queue for BFS. Which implementation would you choose? You need a...

Java Collections — Coding-Based Interview Notes 1. Queue for BFS — which...

How would you implement an LRU cache? How would you design a thread-safe i...

1. How would you implement an LRU Cache? For a simple LRU cache, use...



i will give you the java question you have to answer it according to 5 yr of exp in jav



Absolutely. 👍

Give me any **Java interview question**, and I'll answer it at the level expected from a **5-year experienced Java/Spring Boot developer**, especially for **product-based companies like Amazon, Netflix, Google, Microsoft, etc.**

For each question, I'll structure the answer like this:

1. **Interview-ready answer** — what you should actually say to the interviewer.
2. **Concept explanation** — easy-to-understand explanation so you genuinely understand it.
3. **Internal working** — JVM / memory / threads / collections internals wherever relevant.
4. **Code example** — practical Java code when applicable.
5. **Real-world usage** — how it is used in a production application.
6. **5-year experience perspective** — what an interviewer expects you to know beyond the basic definition.
7. **Follow-up questions** — likely questions the interviewer may ask next, with answers.
8. **Common mistakes** — things you should avoid saying in an interview.

I'll also help you **represent the answer properly in an interview**, rather than just giving a textbook definition.

For example, instead of:

"HashMap stores key-value pairs and uses hashing."

I would help you answer more like:

"HashMap is a non-thread-safe key-value data structure. Internally, it maintains an array of buckets, and the bucket is determined using the key's hash. In modern Java, when multiple keys fall into the same bucket, entries are initially maintained as a linked structure, and under certain conditions the bucket can be converted into a balanced tree to improve lookup performance. Average `get` and `put` are $O(1)$, although collisions can affect this..."

Then I can go deeper into **hash calculation → bucket index → collision → treeification → resizing → load factor → Java 8+ implementation**, depending on what the interviewer asks.

Send me your first Java question.



Why is Java pass-by-value even when we pass an object?



This is a **very common 5-year Java interview question**. The key is to clearly distinguish between **passing an object** and **passing the reference value**.

Interview-ready answer

Java is always pass-by-value. When we pass an object to a method, Java does not pass the actual object or pass the reference itself by reference. Instead, it passes a **copy of the reference value**.

Both the original reference and the copied reference point to the **same object in the heap**. Therefore, if the method modifies the object's fields through that copied reference, the changes are visible to the caller.

However, if we reassign the parameter to a new object, only the local copy of the reference changes. The caller's reference remains unchanged.

So the important distinction is: **Java passes the reference value by value.**

Let's understand it step-by-step

Consider:

```
class Employee {
    String name;
}

public class Test {

    static void change(Employee e) {
        e.name = "Rahul";
    }

    public static void main(String[] args) {

        Employee emp = new Employee();
        emp.name = "Manoj";

        change(emp);
    }
}
```

```

        System.out.println(emp.name);
    }
}

```

Output:

Rahul

You might think:

"The object was passed by reference."

That's not technically correct in Java.

What actually happens?

Initially:

```

main()

emp
|
| reference value
↓
+-----+
| Employee |
| name = Manoj |
+-----+
      Heap

```

When you call:

```
change(emp);
```

Java copies the **reference value** stored in `emp`.

Now:

```

main()                                change()

emp                                    e
|                                    |
|                                    |
└──┬──────────┘                    └──┬──────────┘
    ↓                                ↓

```

```
+-----+
| Employee |
| name = Manoj |
+-----+
      Heap
```

There are **two reference variables**:

```
emp
e
```

But both contain a reference to the **same Employee object**.

Therefore:

```
e.name = "Rahul";
```

modifies the same object.

So `emp.name` also becomes `"Rahul"`.

The important proof: reassignment

Now consider this:

```
static void change(Employee e) {

    e = new Employee();
    e.name = "Rahul";
}
```

And:

```
Employee emp = new Employee();
emp.name = "Manoj";

change(emp);

System.out.println(emp.name);
```

Output:

```
Manoj
```

Why?

Before method call:

```
main()

emp
|
↓
Object A
name = Manoj
```

During method call:

```
main()                change()

emp                    e
|                      |
└──┬──────────┘      └──┬──────────┘
    ↓                  ↓
  Object A            Object A
  name=Manoj          name=Manoj
```

Remember:

`e` is a **copy of the reference value**.

Now:

```
e = new Employee();
```

We change only the local variable `e`.

```
main()                change()

emp                    e
|                      |
↓                      ↓
Object A              Object B
Manoj                 Rahul
```

The caller's `emp` still points to **Object A**.

Therefore:

```
System.out.println(emp.name);
```

prints:

```
Manoj
```

The easiest way to remember it

Think of the reference as an **address/value that tells Java where the object is**.

Suppose:

```
Employee emp = new Employee();
```

Conceptually:

```
emp = 0x100
```

And:

```
0x100 → Employee object
```

When you call:

```
change(emp);
```

Java does:

```
e = 0x100
```

So:

```
emp = 0x100  
e    = 0x100
```

Both point to the same object.

But they are **two separate variables**.

If you do:

```
e.name = "Rahul";
```

you're modifying the object at `0x100`.

But if you do:

```
e = 0x200;
```

you're only changing the local variable `e`.

`emp` is still:

```
emp = 0x100
```

That's why Java is **pass-by-value**.

Primitive vs Object

This comparison is excellent to give in an interview:

Passing	What gets copied?	Can method change caller's variable?
<code>int</code>	Value of <code>int</code>	✗ No
<code>double</code>	Value of <code>double</code>	✗ No
<code>Employee</code> object	Value of the reference	✗ Cannot reassign caller's reference
Object fields	Same underlying object	✓ Can modify fields

Example with primitive:

```
static void change(int x) {  
    x = 20;  
}  
  
int a = 10;  
  
change(a);  
  
System.out.println(a);
```

Output:

```
10
```

Because:

```
a = 10

x = 10    // copy
```

Then:

```
x = 20
```

doesn't affect `a`.

With an object:

```
static void change(Employee e) {
    e.name = "Rahul";
}
```

`e` is still a copied value, but that copied value points to the same object.

A very important interview statement

If the interviewer asks:

"So does Java pass objects by reference?"

Don't say:

Yes, objects are passed by reference.

Instead say:

No. Java is strictly pass-by-value. For objects, the value being passed is the reference to the object. Therefore, a copy of the reference is passed. Both references can access the same object, which is why mutations to the object's state are visible to the caller, but reassignment of the parameter does not affect the caller's reference.

That's the **5-year-experience-level** answer.

One-line memory trick

Java doesn't pass the object by reference; Java passes the object's reference value by value.



just give me the answer like a note not that much explanation



Why is Java pass-by-value even when we pass an object?

Interview Answer:

Java is **always pass-by-value**. When we pass an object to a method, Java passes a **copy of the object's reference value**, not the actual object and not the reference itself by reference.

Both the original reference and the copied reference point to the **same object in the heap**. Therefore, changes made to the object's fields are visible outside the method.

However, if we **reassign the method parameter** to a new object, only the copied reference changes. The original reference remains unchanged.

```
class Employee {
    String name;
}

void change(Employee e) {
    e.name = "Rahul";    // Changes the same object
}

void reassign(Employee e) {
    e = new Employee();  // Only changes local reference
}
```

Key Point

Object passing → Copy of reference value → Both point to same object

So the technically correct statement is:

Java passes the reference to an object by value, meaning a copy of the reference is passed. Java never passes objects by reference.



What exactly happens when `new Employee()` is executed?
How does Garbage Collection determine whether an object can be collected?
Can Java have memory leaks even though it has Garbage Collection?



1. What exactly happens when `new Employee()` is executed?

Interview Answer:

When `new Employee()` is executed:

1. JVM checks whether the `Employee` class is loaded and initialized.
2. Memory is allocated for the new object, typically in the **Heap**.
3. Instance variables get their **default values**.
4. The constructor is executed.
5. A **reference to the newly created object** is returned and can be stored in a reference variable.

```
Employee emp = new Employee();
```

Conceptually:

Stack		Heap
-----		-----
emp	—————>	Employee Object
		name = null
		age = 0

Important: The reference variable `emp` is local and typically stored in the stack frame, while the actual `Employee` object is stored in the heap.

2. How does Garbage Collection determine whether an object can be collected?

Interview Answer:

Garbage Collection primarily determines whether an object is **reachable from GC roots**.

If an object is no longer reachable through any chain of references from GC roots, it becomes **eligible for garbage collection**.

Common GC Roots

- Local variables in active stack frames
- Static fields
- Active threads
- JNI references
- Certain JVM-internal references

Example:

```
Employee emp = new Employee();  
  
emp = null;
```

Now, if there is no other reference to the `Employee` object:

```
GC Root  
  |  
  X  
Employee Object
```

The object is **unreachable and eligible for GC**.

Important Point

Eligible for GC does not mean immediately deleted. The JVM decides when and how garbage collection actually occurs.

Modern collectors such as **G1, ZGC, and Shenandoah** use more sophisticated algorithms, but the fundamental concept is **object reachability**.

3. Can Java have memory leaks even though it has Garbage Collection?

Yes.

Garbage Collection removes objects that are **unreachable**. It cannot remove objects that are still reachable but **no longer needed by the application**.

Example:

```
static List<Employee> employees = new ArrayList<>();

void addEmployee(Employee e) {
    employees.add(e);
}
```

If this list keeps growing and the application never removes old employees:

```
GC Root
  ↓
static employees
  ↓
Employee 1
Employee 2
Employee 3
Employee 4
...
```

All these objects are still reachable through the static list, so GC **cannot collect them**, even if the application no longer needs them.

Common causes of memory leaks in Java

- Static collections holding unnecessary objects
- Objects stored in caches without proper eviction
- Listeners/callbacks not deregistered
- `ThreadLocal` misuse
- Long-lived collections continuously growing
- Incorrectly managed resources

Key Point

Garbage Collection prevents many types of manual-memory-management problems, but it does not prevent logical memory leaks. If unnecessary objects are still reachable, GC considers them alive and cannot reclaim their memory.



What is the Object class in Java?
What is equals()?
What is the contract of equals()?
What is hashCode()?
What is the equals() / hashCode() contract?
Why do HashMap and HashSet need both equals() and hashCode()?
What happens if I override equals() but not hashCode()?
How do you correctly implement equals() and hashCode()?
Why is immutability important for HashMap keys?



Java `Object`, `equals()`, `hashCode()` & Immutability — Interview Notes

1. What is the `Object` class in Java?

`Object` is the **root class of the Java class hierarchy**. Every Java class directly or indirectly extends `Object`.

```
class Employee {  
}
```

is conceptually:

```
class Employee extends Object {  
}
```

Important methods provided by `Object` :

- `equals()`
- `hashCode()`
- `toString()`
- `getClass()`
- `clone()`
- `wait()`
- `notify()`
- `notifyAll()`

Interview point: Every object in Java inherits these methods from `Object` unless they are overridden.

2. What is `equals()` ?

`equals()` is used to determine **logical equality** between two objects.

Default implementation from `Object` essentially compares whether both references refer to the **same object**.

```
Employee e1 = new Employee();
Employee e2 = new Employee();

e1.equals(e2);    // false
```

If we override `equals()`, we can define equality based on business fields.

```
e1.equals(e2);    // true
```

if both employees have the same logical identity.

3. What is the contract of `equals()` ?

`equals()` should satisfy these properties:

Reflexive

```
x.equals(x) == true
```

Symmetric

```
x.equals(y) == y.equals(x)
```

Transitive

If:

```
x.equals(y)
y.equals(z)
```

then:

```
x.equals(z)
```

Consistent

Repeated calls should return the same result as long as the relevant object state hasn't changed.

Non-null

```
x.equals(null) == false
```

Interview point: Breaking these rules can cause unexpected behavior in collections such as `HashMap`, `HashSet`, etc.

4. What is `hashCode()` ?

`hashCode()` returns an integer value representing an object's hash value.

```
int hash = employee.hashCode();
```

It is heavily used by **hash-based collections**, such as:

- `HashMap`
- `HashSet`
- `ConcurrentHashMap`

The hash value helps determine **which bucket** an object should be placed into.

5. What is the `equals()` / `hashCode()` contract?

The most important rule is:

If two objects are equal according to `equals()`, they must return the same `hashCode()`.

```
a.equals(b) == true
```

must imply:

```
a.hashCode() == b.hashCode()
```

But the reverse is **not required**.

Two objects can have the same hash code but still not be equal.

```
Same hashCode
  ↓
Possible collision
  ↓
equals() decides actual equality
```

6. Why do `HashMap` and `HashSet` need both `equals()` and `hashCode()` ?

Because they use **hashCode** to quickly locate the bucket and **equals()** to determine **actual equality**.

Conceptually:

```
key
  ↓
hashCode()
  ↓
bucket
  ↓
equals()
  ↓
matching key?
```

For example:

```
Map<Employee, String> map = new HashMap<>();

map.put(e1, "Developer");
```

When searching:

```
map.get(e2);
```

`HashMap` approximately does:

```
e2.hashCode()
  ↓
find bucket
```

↓
compare keys using equals()
↓
find matching key

Interview point:

`hashCode()` is used for efficient bucket lookup, while `equals()` is used to verify logical equality among candidates in that bucket.

7. What happens if I override `equals()` but not `hashCode()` ?

This can break hash-based collections.

Example:

```
class Employee {  
  
    int id;  
  
    @Override  
    public boolean equals(Object obj) {  
        Employee e = (Employee) obj;  
        return this.id == e.id;  
    }  
}
```

Suppose:

```
Employee e1 = new Employee(101);  
Employee e2 = new Employee(101);  
  
System.out.println(e1.equals(e2)); // true
```

But if `hashCode()` is not overridden, the two objects can have **different hash codes**.

Then:

```
e1 → hashCode A → Bucket 1  
  
e2 → hashCode B → Bucket 2
```

Even though:

```
e1.equals(e2) == true
```

HashMap/HashSet may not find the object correctly.

Interview answer

If I override `equals()` but not `hashCode()`, I violate the `equals()` / `hashCode()` contract. Equal objects can have different hash codes, causing incorrect behavior in `HashMap`, `HashSet`, and other hash-based collections.

8. How do you correctly implement `equals()` and `hashCode()`?

Both methods should use the **same fields that define logical equality**.

Example:

```
class Employee {  
  
    private int id;  
    private String name;  
  
    @Override  
    public boolean equals(Object obj) {  
  
        if (this == obj)  
            return true;  
  
        if (obj == null || getClass() != obj.getClass())  
            return false;  
  
        Employee other = (Employee) obj;  
  
        return id == other.id &&  
            Objects.equals(name, other.name);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(id, name);  
    }  
}
```

Import:

```
import java.util.Objects;
```

Rule

If these fields are used in `equals()` :

```
id + name
```

they should also be used consistently in `hashCode()` :

```
id + name
```

Important: Don't use mutable fields for equality/hash code if the object is going to be used as a `HashMap` key.

9. Why is immutability important for `HashMap` keys?

This is a **very important product-company interview question**.

Consider:

```
Map<Employee, String> map = new HashMap<>();

Employee e = new Employee(101);

map.put(e, "Developer");
```

Suppose `Employee.id` is used in `equals()` and `hashCode()` .

Initially:

```
id = 101
  ↓
hashCode = X
  ↓
Bucket X
```

Now we modify the key:

```
e.setId(200);
```

Its hash code may now change:

```
id = 200
  ↓
hashCode = Y
```

But the object is still physically stored in the bucket determined using the **old hash**.

So:

```
map.get(e);
```

may fail to find the value.

Therefore:

HashMap keys should ideally be immutable.

If the fields used by `equals()` and `hashCode()` change after insertion, the key's hash location can effectively become inconsistent with where it was stored.

Best examples of immutable keys

```
String
Integer
Long
UUID
```

And custom immutable classes can also be used safely as keys.

★ 5-Year Experience Summary

Remember these points for the interview:

```
Object
  ↓
Root class of Java hierarchy

equals()
  ↓
Checks logical equality

hashCode()
  ↓
Helps hash-based collections locate buckets
```

`equals + hashCode`

↓

Equal objects MUST have same hashCode

HashMap

↓

`hashCode()` → bucket

`equals()` → actual key comparison

Override `equals()` only

↓

Breaks equals/hashCode contract

Mutable HashMap key

↓

Changing equality fields can make the key difficult/impossible to retrieve correctly

One-line answer to remember

`hashCode()` ***narrows down where to look, and*** `equals()` ***confirms whether it is actually the same logical object.***



How do you create an immutable class?
Why is String immutable?
What is the String Pool?
What happens with new String("Java")?
What does intern() do?
Why can String safely be used as a HashMap key?
What happens with String a = "Hello"; String b = "Hel" + "lo";?
What happens when String concatenation involves a runtime variable?
Why don't we use String for repeated concatenation?
What is the difference between StringBuilder and StringBuffer?
How does HashSet determine whether two objects are duplicates?



Immutability, String & HashSet — Interview Notes

1. How do you create an immutable class?

An immutable class is a class whose **state cannot be changed after object creation**.

Rules:

- Make the class `final` so it cannot be extended.
- Make fields `private` and `final`.
- Initialize fields through the constructor.
- Don't provide setters.
- For mutable objects, create **defensive copies** in the constructor and getters.

```
final class Employee {  
  
    private final int id;  
    private final String name;  
  
    public Employee(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

```

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}

```

Key point: Immutability means the object's **state cannot change after construction**.

2. Why is `String` immutable?

`String` is immutable because its internal value cannot be changed after creation.

Important reasons:

- **String Pool** — strings can be safely shared between multiple references.
- **Security** — strings are commonly used for file paths, URLs, class names, credentials, etc.
- **Thread safety** — immutable strings can safely be shared between threads.
- **HashMap keys** — their hash code doesn't change after creation.
- **Performance** — JVM can cache/reuse strings safely.

```

String s = "Java";
s = "Python";

```

The original `"Java"` object is **not modified**. `s` simply points to another `String` object.

3. What is the String Pool?

The **String Pool** is a special pool maintained by the JVM for sharing string literals.

```

String a = "Java";
String b = "Java";

```

Both references point to the **same pooled String object**.

```

a ———┐
      │
      ↓
  "Java" ← String Pool

```



Therefore:

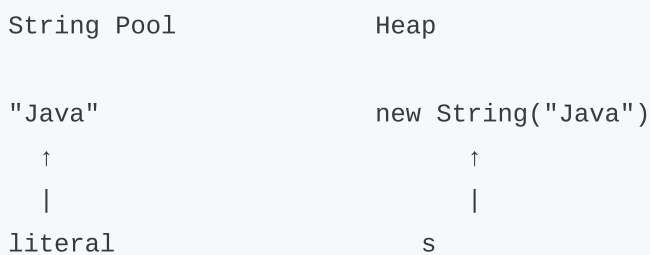
```
a == b      // true
a.equals(b) // true
```

4. What happens with `new String("Java")` ?

```
String s = new String("Java");
```

The literal `"Java"` is obtained from the String Pool, but `new String()` creates a **new String object**.

Conceptually:



Therefore:

```
String a = "Java";
String b = new String("Java");

a == b      // false
a.equals(b) // true
```

`==` compares references, while `equals()` compares String content.

5. What does `intern()` do?

`intern()` returns the **canonical pooled representation** of a String.

```
String a = new String("Java");
String b = a.intern();

String c = "Java";
```

```
System.out.println(b == c); // true
```

If the String already exists in the pool, `intern()` returns that pooled reference.

If it doesn't exist, the String is added to the pool and that pooled reference is returned.

6. Why can String safely be used as a HashMap key?

Because `String` is **immutable**.

```
Map<String, Integer> map = new HashMap<>();  
  
map.put("Java", 100);
```

The String's value cannot change after insertion, so its `hashCode()` remains consistent.

Therefore:

```
String key  
  ↓  
hashCode()  
  ↓  
Bucket
```

The key won't suddenly change its hash value and become difficult to retrieve.

7. What happens with:

```
String a = "Hello";  
String b = "Hel" + "lo";
```

Because both parts are **compile-time constants**, the compiler performs **constant folding**.

Conceptually:

```
String b = "Hello";
```

Both refer to the same String Pool object.

Therefore:

```
a == b          // true  
a.equals(b)     // true
```

8. What happens when String concatenation involves a runtime variable?

Example:

```
String x = "Hel";  
String a = "Hello";  
String b = x + "lo";
```

Because `x` is determined at **runtime**, `"Hel" + "lo"` cannot be treated as a compile-time constant expression.

The concatenation is performed at runtime, and `b` will generally refer to a newly created result rather than the pooled literal `"Hello"`.

```
a == b          // false  
a.equals(b)     // true
```

Interview point

Compile-time concatenation → compiler can fold it

Runtime concatenation → concatenation happens at runtime

9. Why don't we use String for repeated concatenation?

Because `String` is immutable.

```
String result = "";  
  
for (int i = 0; i < 1000; i++) {  
    result = result + i;  
}
```

Every concatenation can create a **new String object**, causing unnecessary object creation and copying.

For repeated modifications, use:

```
StringBuilder
```

Example:

```

StringBuilder sb = new StringBuilder();

for (int i = 0; i < 1000; i++) {
    sb.append(i);
}

String result = sb.toString();

```

This is generally much more efficient for repeated concatenation.

10. Difference between `StringBuilder` and `StringBuffer`

Feature	<code>StringBuilder</code>	<code>StringBuffer</code>
Mutable	Yes	Yes
Thread-safe	No	Yes
Synchronization	No	Methods are synchronized
Performance	Generally faster	Generally slower
Introduced	Java 5	Java 1.0
Typical use	Single-threaded code	Shared mutable string across threads

Interview answer

`StringBuilder` is preferred for most cases because it is not synchronized and therefore generally provides better performance. `StringBuffer` is synchronized and is useful when the same mutable buffer must be safely accessed by multiple threads.

11. How does `HashSet` determine whether two objects are duplicates?

`HashSet` internally uses a `HashMap`.

When adding an object:

```
set.add(employee);
```

Conceptually:

```
object
  ↓
hashCode()
  ↓
find bucket
  ↓
equals()
  ↓
duplicate?
```

Case 1: Different hash codes

The objects are treated as different candidates.

Case 2: Same hash code

A **hash collision** may have occurred, so `HashSet` uses `equals()` to determine whether the objects are actually equal.

```
hashCode() same
  ↓
equals()
  ↓
├── true
│   ↓
│   duplicate
└── false
    ↓
    different objects
```

Important rule

For `HashSet`, `hashCode()` helps locate the bucket, and `equals()` determines whether an existing object in that bucket is logically equal.

So if you create custom objects for a `HashSet`, you should correctly override **both** `equals()` and `hashCode()`.



What happens when "Java" and new String("Java") are added to a HashSet?

What is the difference between == and equals() for Strings?



1. What happens when "Java" and new String("Java") are added to a HashSet ?

```
Set<String> set = new HashSet<>();  
  
set.add("Java");  
set.add(new String("Java"));
```

The `HashSet` will contain **only one element**.

Why?

```
"Java"  
  ↓  
hashCode()  
  ↓  
same hash code  
  ↓  
equals()  
  ↓  
true  
  ↓  
Duplicate → not added
```

`String` overrides both `hashCode()` and `equals()` based on its **content**.

So:

```
"Java".equals(new String("Java")) // true
```

Therefore:

```
set.size(); // 1
```

Interview point

`HashSet` does not care whether the two `String` objects are different objects in memory. It uses `hashCode()` and `equals()` to determine logical equality. Since both `Strings` contain `"Java"`, the second one is considered a duplicate.

2. Difference between `==` and `equals()` for `Strings`

`==`

`==` compares the **references**, meaning whether two variables point to the **same `String` object**.

```
String a = "Java";
String b = "Java";

System.out.println(a == b); // true
```

Because both refer to the same `String` Pool object.

But:

```
String a = "Java";
String b = new String("Java");

System.out.println(a == b); // false
```

They contain the same text but are different objects.

`equals()`

`equals()` compares the **content/value** of the `Strings`.

```
String a = "Java";
String b = new String("Java");

System.out.println(a.equals(b)); // true
```

Quick comparison

	<code>==</code>	<code>equals()</code>
Compares	Reference	Content
Same object required?	Yes	No

	<code>==</code>	<code>equals()</code>
<code>"Java"</code> vs <code>new String("Java")</code>	<code>false</code>	<code>true</code>
Recommended for String comparison	✗	✓

Interview one-liner

For Strings, `==` checks whether two references point to the same object, whereas `equals()` checks whether the two String objects contain the same sequence of characters.



How does HashMap work internally?
What happens when map.put(key, value) is called?
How is the bucket calculated?
What is hashing and hash collision?
What happens when two keys have the same hash?
Why are equals() and hashCode() both required?
What happens when a HashMap exceeds its load factor?
What is resizing/rehashing?
Why is the default load factor 0.75?
Why is HashMap capacity generally a power of 2?



HashMap Internals — 5-Year Interview Notes

1. How does `HashMap` work internally?

`HashMap` stores data as **key-value pairs**.

Internally, it uses an **array of buckets**. Each bucket can contain multiple entries when hash collisions occur.

Conceptually:

```
HashMap
  |
  ↓
Bucket Array
[0] → Entry
[1] → Entry → Entry
[2]
[3] → Entry
...
```

Each entry contains approximately:

```
key
value
hash
next
```

In modern Java, if too many entries accumulate in a bucket, the bucket can be converted from a linked structure to a **Red-Black Tree** under certain conditions.

2. What happens when `map.put(key, value)` is called?

Example:

```
map.put("Java", 100);
```

Conceptually, HashMap performs these steps:

1. Calculate key's hashCode()
↓
2. Spread/mix the hash
↓
3. Calculate bucket index
↓
4. Check bucket
↓
5. If empty → insert
↓
6. If occupied → compare hash + equals()
↓
7. Same key → replace value
Different key → collision handling
↓
8. Check whether resize is required

3. How is the bucket calculated?

For a HashMap with capacity `n`, the bucket index is conceptually calculated as:

```
index = (n - 1) & hash;
```

For example, if:

```
capacity = 16
```

then:

```
index = (16 - 1) & hash  
       = 15 & hash
```

HashMap also performs a hash-spreading operation before calculating the bucket.

Conceptually in Java 8+:

```
int h = key.hashCode();  
int hash = h ^ (h >>> 16);
```

Then:

```
index = (capacity - 1) & hash;
```

Why not simply `% capacity`?

Because when capacity is a **power of 2**, bitwise `&` can efficiently calculate the index.

4. What is hashing and hash collision?

Hashing

Hashing converts a key into an integer hash value.

```
key.hashCode()
```

Example:

```
"Java"  
  ↓  
hashCode()  
  ↓  
some integer  
  ↓  
bucket index
```

Hash Collision

A collision occurs when **different keys end up in the same bucket**.

For example:

```
Key A → Bucket 5  
Key B → Bucket 5
```

Even if their hash codes are different, they can still map to the same bucket because the bucket index is derived from the hash and capacity.

5. What happens when two keys have the same hash?

HashMap stores both entries in the same bucket.

Example:

```
Bucket 5

Key A → Value A
  ↓
Key B → Value B
```

HashMap then uses:

```
hash comparison
  +
equals()
```

to determine whether the keys are actually the same.

If:

```
key1.equals(key2)
```

is `true`, HashMap considers them the **same key** and updates the existing value.

If `equals()` is `false`, they are different keys and both entries are retained.

6. Why are `equals()` and `hashCode()` both required?

Because they perform different jobs.

```
hashCode()
  ↓
Find the probable bucket quickly

equals()
  ↓
Confirm whether keys are actually equal
```

Example:

```
Key A → hashCode → Bucket 5
Key B → hashCode → Bucket 5
                ↓
                equals()
```

↓
Same or different?

Contract

If:

```
key1.equals(key2) == true
```

then:

```
key1.hashCode() == key2.hashCode()
```

must also be true.

7. What happens when a HashMap exceeds its load factor?

HashMap has:

```
capacity  
load factor  
threshold
```

The resize threshold is approximately:

```
threshold = capacity × loadFactor
```

Default:

```
capacity = 16  
load factor = 0.75  
  
threshold = 16 × 0.75  
           = 12
```

When the number of entries crosses the threshold, HashMap **resizes**.

8. What is resizing / rehashing?

Suppose the current capacity is:

```
16
```

When resizing occurs, it generally grows to:

32

Then the entries need to be redistributed according to the new capacity.

Conceptually:

Before:

Capacity = 16

Bucket 3 → A

Bucket 7 → B

Bucket 15 → C

Resize

Capacity = 32

Bucket 3 → A

Bucket 7 → B

Bucket 15 → C

...

The bucket positions can change because the capacity changed.

Important interview distinction

Resizing = increasing the internal bucket array capacity.

Rehashing/redistribution = determining where existing entries belong under the new capacity.

In modern Java implementations, the redistribution during resize is optimized and does not simply mean calling `hashCode()` again on every key.

9. Why is the default load factor 0.75?

0.75 is a **trade-off between memory usage and performance**.

Lower load factor

Fewer entries per bucket
↓
Fewer collisions
↓
Better lookup performance
↓
More memory required

Higher load factor

More entries per bucket
↓
More collisions
↓
Potentially slower operations
↓
Better memory utilization

0.75 provides a practical balance between **space** and **lookup performance**.

10. Why is HashMap capacity generally a power of 2?

HashMap uses:

```
index = (capacity - 1) & hash;
```

This works efficiently when capacity is a power of 2.

For example:

```
capacity = 16  
capacity - 1 = 15
```

Binary:

```
15 = 1111
```

Therefore:

```
hash & 1111
```

efficiently extracts the lower bits needed for the bucket index.

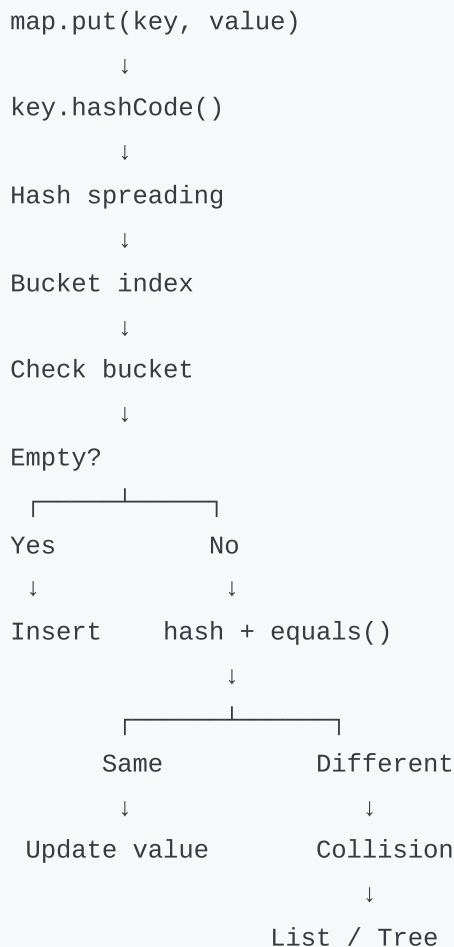
Power-of-two capacity also makes resizing from:

16 → 32 → 64 → 128

efficient and allows the implementation to efficiently determine how entries move between buckets.

★ 5-Year Interview Summary

Remember this flow:



And remember these numbers:

Default initial capacity = 16
Default load factor = 0.75
Default threshold = $16 \times 0.75 = 12$
Resize = generally doubles capacity

Most important interview line

HashMap uses `hashCode()` to determine the bucket and `equals()` to identify the exact key. When the number of entries crosses the threshold ($\text{capacity} \times$

`loadFactor`), ***HashMap*** ***resizes its bucket array and redistributes the existing entries.***



What changed in Java 8 HashMap?
Why does HashMap convert a bucket into a Red-Black Tree?
What is treeification?
What is TREEIFY_THRESHOLD?
What is UNTREEIFY_THRESHOLD?
What happens if a mutable object is used as a HashMap key?
Why is HashMap not thread-safe?
HashMap vs ConcurrentHashMap
HashMap vs Hashtable
How would you design a custom HashMap?



HashMap Advanced — 5-Year Interview Notes

1. What changed in Java 8 `HashMap` ?

The major change was **collision handling**.

Before Java 8

A bucket generally used a **linked list**:

```
Bucket
  ↓
Node → Node → Node → Node
```

Searching could degrade toward:

$O(n)$

Java 8+

When a bucket becomes heavily populated, it can be converted into a **Red-Black Tree**:

```
Bucket
  ↓
Red-Black Tree
    8
   / \
  4   12
```

This improves worst-case lookup within a heavily-collided bucket from approximately:

$$O(n) \rightarrow O(\log n)$$

Other important changes include improved resize/redistribution logic and the use of `Node` / `TreeNode` structures internally.

2. Why does HashMap convert a bucket into a Red-Black Tree?

To prevent poor performance caused by **many hash collisions**.

With a linked list:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$$

Searching can become:

$$O(n)$$

With a Red-Black Tree:

```
      C
     / \
    B   E
     / \
    D   F
```

Searching becomes approximately:

$$O(\log n)$$

Interview point

Treeification protects HashMap from severe collision scenarios where a bucket would otherwise behave like a long linked list.

3. What is treeification?

Treeification is the process of converting a bucket's linked-list representation into a **Red-Black Tree**.

Conceptually:

Before:

Bucket → A → B → C → D → E → F → G → H

After:

Bucket → Red-Black Tree



This happens only when certain conditions are satisfied.

4. What is `TREEIFY_THRESHOLD` ?

In the standard Java 8+ `HashMap` implementation:

```
TREEIFY_THRESHOLD = 8
```

It means that when a bucket reaches approximately **8 nodes**, `HashMap` may attempt to treeify that bucket.

But there's an important condition:

```
MIN_TREEIFY_CAPACITY = 64
```

If the table capacity is **less than 64**, `HashMap` generally prefers to **resize the table instead of treeifying**.

So don't simply say:

"At 8 entries it becomes a tree."

Better interview answer:

When a bucket reaches the treeification threshold of 8 nodes, `HashMap` may convert it to a Red-Black Tree, provided the table has reached the minimum treeify capacity of 64; otherwise it generally resizes first.

5. What is `UNTREEIFY_THRESHOLD` ?

```
UNTREEIFY_THRESHOLD = 6
```

When a tree bucket becomes sufficiently small, HashMap can convert the tree back into a linked-list representation.

This is called **untreeification**.

```
Tree
  ↓
entries decrease
  ↓
≤ approximately 6
  ↓
Linked List
```

Important nuance

Don't present 6 as a strict "every time size becomes 6, immediately convert" rule. The actual conversion can depend on the operation and implementation path.

6. What happens if a mutable object is used as a HashMap key?

This is dangerous.

Example:

```
Employee emp = new Employee(101);

Map<Employee, String> map = new HashMap<>();
map.put(emp, "Developer");

emp.setId(200);

map.get(emp);
```

If `id` is used in `hashCode()` and `equals()`, changing it changes the key's hash code.

Conceptually:

```
Before:

id = 101
  ↓
hash = X
```

↓
Bucket X

After mutation:

id = 200
↓
hash = Y

But the entry was originally placed based on the old hash.

Therefore, retrieval can fail.

Interview answer

A mutable key can become effectively "lost" inside a HashMap if fields used by `equals()` / `hashCode()` are modified after insertion. Therefore, HashMap keys should preferably be immutable.

7. Why is HashMap not thread-safe?

`HashMap` does not provide synchronization for concurrent access.

If multiple threads modify the same HashMap simultaneously, operations such as:

```
put()  
remove()  
resize()
```

can interfere with each other.

This can lead to:

- Lost updates
- Inconsistent state
- Visibility problems
- Incorrect results

Therefore, for concurrent access, use an appropriate concurrent collection such as:

```
ConcurrentHashMap
```

or externally synchronize access when appropriate.

8. HashMap vs ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap
Thread-safe	✗	✓
Null key	Allows one	✗
Null values	Allows	✗
Concurrent access	Not safe	Designed for it
Performance	Good single-threaded	Optimized concurrent access
Typical use	Local/non-shared data	Shared concurrent data

Interview point

`ConcurrentHashMap` does **not** simply put one big lock around the entire map.

Modern implementations use techniques such as:

- CAS
- Fine-grained synchronization
- Volatile reads/writes
- Bucket-level coordination

to allow a high degree of concurrent access.

9. HashMap vs Hashtable

Feature	HashMap	Hashtable
Thread-safe	✗	✓
Synchronization	No	Synchronized methods
Null key	Yes	No
Null value	Yes	No
Performance	Generally better	Generally lower under contention
Legacy	No	Yes
Recommended for new code	Yes	Generally no

Interview answer

`Hashtable` is a legacy synchronized collection. For concurrent applications, `ConcurrentHashMap` is generally preferred over `Hashtable` because it provides better concurrency characteristics.

10. How would you design a custom HashMap?

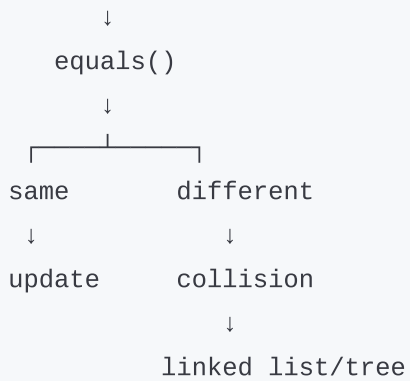
For a 5-year interview, explain the **core design first**, then discuss production-level improvements.

Basic structure

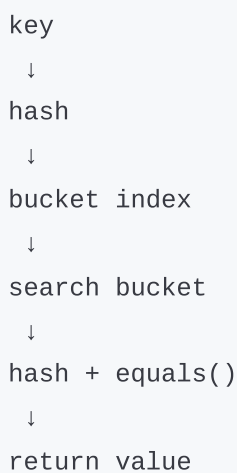
```
class MyHashMap<K, V> {  
  
    static class Node<K, V> {  
        K key;  
        V value;  
        int hash;  
        Node<K, V> next;  
    }  
  
    private Node<K, V>[] table;  
    private int size;  
    private float loadFactor = 0.75f;  
}
```

`put()` flow

```
key  
↓  
hashCode()  
↓  
hash spreading  
↓  
bucket index  
↓  
bucket empty?  
├─ Yes → insert  
└─ No  
    ↓  
    compare hash
```



`get()` flow



Resize

When:

$\text{size} > \text{capacity} \times \text{loadFactor}$

increase capacity, typically:

$16 \rightarrow 32 \rightarrow 64 \rightarrow 128$

and redistribute the entries.

Advanced improvements

For a production-quality implementation, I would consider:

- Collision handling
- Red-Black Tree conversion

- Resize optimization
- Load factor configuration
- Null-key handling
- Iterator behavior
- Fail-fast behavior
- Generic type safety
- Concurrency strategy
- Memory overhead
- Hash-quality considerations

Interview-ready answer

I would design a custom HashMap using an array of buckets, where each bucket contains nodes holding the key, value, hash and next reference. `put()` calculates the hash and bucket index, then uses `equals()` to detect an existing key or handle collisions. Initially collisions can be handled using linked lists, and for heavily populated buckets I could introduce treeification. Once the size crosses $\text{capacity} \times \text{loadFactor}$, I would resize the table and redistribute the entries. For production-level concurrency, I would use a separate concurrency design rather than simply synchronizing the entire map.



What happens if two keys have the same hashCode?
What happens if hashCode() is same but equals() is false?
What happens if equals() is true but hashCode() is different?
What happens if a HashMap key is mutable?
Can HashMap contain a null key?
Can HashMap contain multiple null values?
Why is HashMap not thread-safe?
What happens if multiple threads modify HashMap simultaneously



HashMap Edge Cases — 5-Year Interview Notes

1. What happens if two keys have the same `hashCode()` ?

They may end up in the **same bucket**.

HashMap then uses `equals()` to determine whether they are actually the same key.

```
Same hashCode()  
    ↓  
Same bucket  
    ↓  
equals()  
  ↓   ↓  
true  false  
  ↓   ↓  
Same key Collision
```

If `equals()` is `false`, both entries can exist.

2. What happens if `hashCode()` is same but `equals()` is false?

This is a **hash collision**.

Both keys can be stored in the same bucket.

```
key1.hashCode() == key2.hashCode() // true  
key1.equals(key2)                  // false
```

HashMap keeps both entries because they are logically different keys.

Important: Same hash code does **not** mean objects are equal.

3. What happens if `equals()` is true but `hashCode()` is different?

This **violates the** `equals()` / `hashCode()` **contract**.

```
key1.equals(key2) == true
key1.hashCode() != key2.hashCode()
```

This can cause incorrect behavior in `HashMap` / `HashSet` .

For example, the two equal keys may be mapped to different buckets, so `HashMap` may fail to find an existing entry.

Rule:

*If `equals()` is true, `hashCode()` **must be the same**.*

4. What happens if a `HashMap` key is mutable?

It can cause the key to become effectively **unretrievable**.

```
Employee e = new Employee(101);

map.put(e, "Developer");

e.setId(200); // id used by hashCode()
```

If the mutation changes `hashCode()` :

```
Before:
id=101 → hash=A → Bucket A

After:
id=200 → hash=B
```

The entry is still located based on the old placement, while lookup uses the new hash.

```
map.get(e); // may return null
```

Interview point: Use immutable objects, such as `String`, as `HashMap` keys whenever possible.

5. Can `HashMap` contain a null key?

Yes.

A `HashMap` can contain **one null key**.

```
Map<String, Integer> map = new HashMap<>();

map.put(null, 100);
```

```
map.put(null, 200);
```

The second operation **replaces** the value associated with the null key.

```
map.size(); // 1
```

6. Can HashMap contain multiple null values?

Yes.

A HashMap can contain **multiple null values**, because values don't have to be unique.

```
map.put("A", null);
map.put("B", null);
map.put("C", null);
```

All three entries can exist.

```
A → null
B → null
C → null
```

Remember:

```
Null keys    → Maximum 1
Null values  → Multiple allowed
```

7. Why is HashMap not thread-safe?

`HashMap` does not provide thread-safe coordination for concurrent modifications.

Multiple threads can simultaneously perform:

```
put()
remove()
resize()
```

without the required synchronization.

This can lead to:

- Lost updates
- Visibility issues
- Inconsistent results
- Race conditions

For shared concurrent access, use:

```
ConcurrentHashMap
```

when its semantics fit the use case.

8. What happens if multiple threads modify HashMap simultaneously?

If multiple threads modify the same `HashMap` without synchronization, the behavior is **not safe or predictable**.

Example:

```
Map<Integer, String> map = new HashMap<>();
```

Then:

```
Thread 1 → put()  
Thread 2 → put()  
Thread 3 → remove()
```

may execute concurrently.

Possible consequences include:

- **Lost updates**
- **Incorrect** `size()`
- **Missing entries**
- **Visibility problems**
- **Race conditions**
- In older Java implementations, concurrent resize could cause severe structural problems; modern Java should not be described as having the old Java-7 infinite-loop behavior.

Interview answer

`HashMap` is not thread-safe because its operations don't provide the synchronization required for concurrent modification. If multiple threads need to access and modify the same map concurrently, I would normally use `ConcurrentHashMap` or explicitly synchronize access depending on the use case.

★ Quick Interview Revision

Same hashCode + equals false

- Collision
- Both keys can exist

Same hashCode + equals true

- Same logical key
- Existing value gets replaced

equals true + hashCode different

- Contract violation
- HashMap/HashSet can behave incorrectly

Mutable key

- hashCode can change
- Retrieval can fail

HashMap

- 1 null key
- Multiple null values
- Not thread-safe

Most important line

`hashCode()` **identifies the bucket;** `equals()` **identifies the exact key within that bucket.**



HashMap vs Hashtable
HashMap vs ConcurrentHashMap
HashMap vs LinkedHashMap
HashMap vs TreeMap
How does ConcurrentHashMap work?
Why doesn't ConcurrentHashMap allow null keys?
How would you make HashMap thread-safe?
What problems can occur if HashMap is modified concurrently?
How would you diagnose poor HashMap performance in production?



HashMap & Map Variants — 5-Year Interview Notes

1. HashMap vs Hashtable

Feature	HashMap	Hashtable
Thread-safe	✗ No	✓ Yes
Synchronization	No	Synchronized methods
Null key	✓ One	✗ No
Null values	✓ Multiple	✗ No
Performance	Generally faster	Generally slower
Legacy	Modern collection	Legacy class
Preferred today	✓ Yes	Usually no

Interview:

`Hashtable` is a legacy synchronized Map. For concurrent applications, I would generally prefer `ConcurrentHashMap` because it provides better concurrency.

2. HashMap vs ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap
Thread-safe	✗	✓

Feature	HashMap	ConcurrentHashMap
Null key	1 allowed	✗
Null values	Allowed	✗
Concurrent reads	Not guaranteed safe with concurrent writes	Designed for concurrent access
Concurrent updates	Not safe	Supported
Performance	Good single-threaded	Optimized for concurrency

Interview:

`ConcurrentHashMap` provides thread-safe access without locking the entire map for every operation, allowing better concurrency than synchronizing a `HashMap` globally.

3. HashMap vs LinkedHashMap

Feature	HashMap	LinkedHashMap
Ordering	No guaranteed iteration order	Maintains insertion/access order
Performance	Slightly better	Slightly more overhead
Internal structure	Hash table	Hash table + linked list
Null key	Yes	Yes
Typical use	General-purpose Map	Need predictable iteration order

Example:

```
Map<Integer, String> map = new LinkedHashMap<>();
```

It can maintain:

Insertion order:

```
1 → A
2 → B
3 → C
```

`LinkedHashMap` can also be configured for **access-order**, which is useful for implementing LRU-style caches.

4. HashMap vs TreeMap

Feature	HashMap	TreeMap
Ordering	No guaranteed order	Sorted order
Internal structure	Hash table	Red-Black Tree
Average <code>get()</code>	$O(1)$	$O(\log n)$
Average <code>put()</code>	$O(1)$	$O(\log n)$
Null key	One allowed	Generally not allowed with natural ordering
Use case	Fast key lookup	Sorted/range-based operations

Example:

```
Map<Integer, String> map = new TreeMap<>();

map.put(30, "C");
map.put(10, "A");
map.put(20, "B");
```

Iteration:

```
10 → A
20 → B
30 → C
```

Interview:

Use `HashMap` when fast key-based lookup is the priority. Use `TreeMap` when keys need to remain sorted or when operations such as range queries are required.

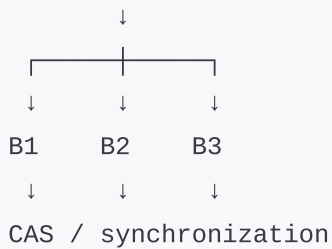
5. How does ConcurrentHashMap work?

Modern `ConcurrentHashMap` uses a combination of **CAS, volatile operations, and synchronized sections around specific buckets/nodes** rather than one global lock for the entire map.

Conceptually:

```
ConcurrentHashMap
↓
```

Bucket array



Multiple threads can operate on different portions of the map concurrently.

For updates, it uses mechanisms such as:

- CAS for certain insertion/update operations
- Fine-grained synchronization when required
- Volatile reads/writes for visibility
- Tree bins for heavily-collided buckets

Important interview point

Don't say:

"ConcurrentHashMap uses segment locking."

That describes the **older Java 7 implementation**. Java 8+ significantly changed the implementation.

6. Why doesn't ConcurrentHashMap allow null keys?

Because `null` creates ambiguity in concurrent operations.

For example:

```
map.get(key);
```

If the result is:

```
null
```

it could mean:

1. Key doesn't exist
2. Key exists but has a null value

`ConcurrentHashMap` avoids this ambiguity by disallowing both:

null key ❌
null value ❌

Interview answer:

`ConcurrentHashMap` disallows null keys and values because in concurrent scenarios a null return value needs to unambiguously mean that there is no mapping for the key.

7. How would you make HashMap thread-safe?

There are several approaches depending on the requirement.

Option 1: `ConcurrentHashMap` — preferred for concurrent access

```
Map<String, Integer> map = new ConcurrentHashMap<>();
```

Option 2: `Collections.synchronizedMap()`

```
Map<String, Integer> map =  
    Collections.synchronizedMap(new HashMap<>());
```

This synchronizes map operations.

For iteration, external synchronization may also be required according to the API contract.

Option 3: Explicit synchronization

```
synchronized (map) {  
    map.put("Java", 100);  
}
```

Interview answer

For a highly concurrent application, I would generally prefer `ConcurrentHashMap`.
`Collections.synchronizedMap()` is suitable when I need a synchronized wrapper and concurrency requirements are simpler.

8. What problems can occur if HashMap is modified concurrently?

Without proper synchronization, you can get:

- Race conditions
- Lost updates
- Missing entries
- Inconsistent results
- Visibility problems
- Incorrect size/count observations
- Unsafe iteration, including `ConcurrentModificationException` in relevant iteration scenarios

Example:

```
Thread 1 → put(A)
Thread 2 → put(B)
Thread 3 → remove(A)
```

Without synchronization, operations can interfere with one another.

Important

Don't use the old statement:

"Concurrent HashMap modification always causes an infinite loop."

That was associated with some **old Java 7 HashMap resize behavior**, not modern Java HashMap.

9. How would you diagnose poor HashMap performance in production?

For a 5-year candidate, don't simply say:

"I will check the HashMap."

Give a systematic approach.

Step 1 — Check application metrics

Look for:

- CPU usage
- Heap usage
- GC activity
- Request latency
- Throughput
- Allocation rate

Step 2 — Check HashMap usage

Look for:

```
Huge HashMaps
↓
High memory consumption
↓
Frequent resizing
↓
Excessive object creation
```

Check whether the initial capacity is appropriate.

Step 3 — Investigate hash collisions

Poorly implemented `hashCode()` can cause many keys to land in the same buckets.

For custom keys, verify:

```
equals()
hashCode()
```

are implemented correctly and distribute hashes reasonably.

Step 4 — Check mutable keys

Look for keys whose fields used by `equals()` / `hashCode()` are modified after insertion.

Step 5 — Take heap/thread profiles

In production-like environments, use profiling/diagnostic tools to identify:

- Large HashMap instances

- Dominant objects
- Allocation hotspots
- GC pressure
- CPU hotspots

Step 6 — Check concurrency

If multiple threads access the same map:

```
HashMap + concurrent modification
      ↓
Potential race conditions
```

Consider whether `ConcurrentHashMap` or another design is appropriate.

Interview-ready answer

I would first identify whether the problem is CPU, memory, GC, or contention related using application metrics and profiling. Then I would inspect the map's size, resizing behavior, key distribution and `hashCode()` implementation. For custom keys, I would verify the `equals()` / `hashCode()` contract and check for mutable keys. If the map is shared across threads, I would also investigate concurrent access and whether `ConcurrentHashMap` is more appropriate.

★ Quick Revision

HashMap

- Fast lookup
- Not thread-safe

Hashtable

- Synchronized
- Legacy
- Avoid for new concurrent designs

LinkedHashMap

- Maintains insertion/access order

TreeMap

- Sorted keys
- $O(\log n)$

ConcurrentHashMap

- Thread-safe
- CAS + fine-grained synchronization
- No null keys/values

Most important interview distinction

HashMap optimizes for general-purpose lookup, LinkedHashMap adds predictable ordering, TreeMap provides sorted-key operations, and ConcurrentHashMap provides safe concurrent access.



What is the Java Collections Framework?
What is the difference between Collection and Collections?
What is the difference between Collection and Map?
Explain the Java Collections hierarchy.
What is the difference between List, Set, Queue, and Map?
When would you choose List vs Set vs Map?
Why doesn't Map extend Collection?
What is Iterable and how is it related to Collections?
What is the difference between Iterator and ListIterator?
What is fail-fast vs fail-safe iteration?



Java Collections Framework — 5-Year Interview Notes

1. What is the Java Collections Framework?

The **Java Collections Framework (JCF)** is a set of **interfaces, implementations, and utility methods** used to store and manipulate groups of objects.

Main interfaces:

```
Iterable
  ↓
Collection
  ├── List
  ├── Set
  └── Queue

Map ← separate hierarchy
```

Common implementations:

```
List → ArrayList, LinkedList
Set → HashSet, LinkedHashSet, TreeSet
Queue → PriorityQueue, ArrayDeque
Map → HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap
```

2. Collection vs Collections

Collection

Collection is an **interface**.

It is the root interface for `List`, `Set`, and `Queue`.

```
Collection<String> c;
```

Collections

`Collections` is a **utility class** containing static methods for working with collections.

```
Collections.sort(list);
Collections.reverse(list);
Collections.unmodifiableList(list);
```

Remember

```
Collection → Interface
Collections → Utility class
```

3. Collection vs Map

`Collection` represents a **group of individual elements**.

```
[A, B, C, D]
```

`Map` represents **key-value mappings**.

```
Key → Value
```

```
101 → Manoj
102 → Rahul
103 → Amit
```

`Map` does **not extend** `Collection`.

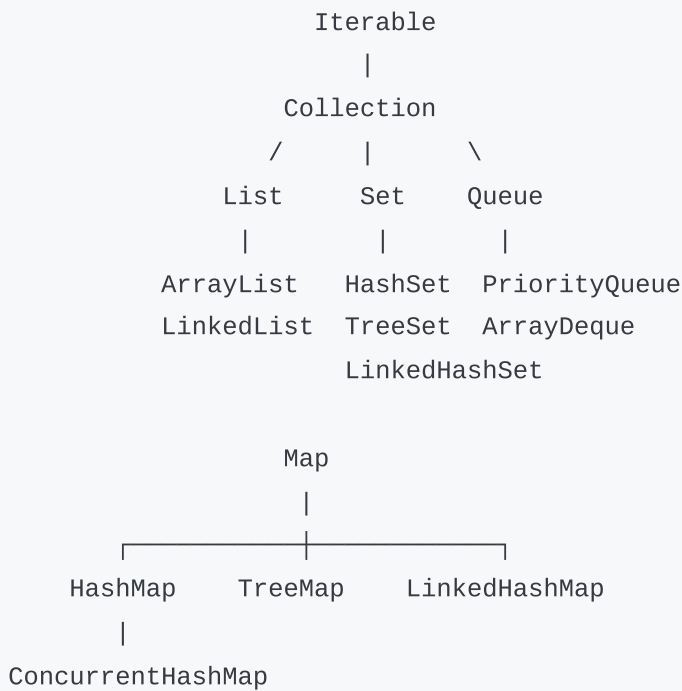
Examples:

```
Collection → List / Set / Queue
```

```
Map → HashMap / TreeMap / LinkedHashMap
```

4. Java Collections Hierarchy

Simplified hierarchy:



Important: **Map is separate from Collection.**

5. Difference between List, Set, Queue and Map

Type	Purpose	Duplicates	Ordering
List	Ordered collection	✅ Allowed	Yes
Set	Unique elements	❌ Not allowed	Depends on implementation
Queue	Processing elements	Usually allowed	Processing order
Map	Key-value mapping	Keys ❌, values ✅	Depends on implementation

Examples:

```

List<String> list = new ArrayList<>();

Set<String> set = new HashSet<>();

Queue<String> queue = new ArrayDeque<>();

Map<Integer, String> map = new HashMap<>();
  
```

6. When would you choose List vs Set vs Map?

Choose `List`

When:

- Duplicates are allowed.
- You care about element order.
- You need index-based access.

```
List<String> users = new ArrayList<>();
```

Choose `Set`

When:

- Elements must be unique.
- You mainly care about membership/uniqueness.

```
Set<String> uniqueUsers = new HashSet<>();
```

Choose `Map`

When:

- You need to associate one value with a key.
- Fast lookup by key is required.

```
Map<Long, Employee> employees = new HashMap<>();
```

Interview example

If I need to store employees in insertion order, I might use `List`. If I need unique employee IDs, I would use `Set`. If I need to find an employee using employee ID, I would use `Map`.

7. Why doesn't Map extend Collection?

Because their data models are fundamentally different.

`Collection` represents:

Individual elements

A

B

C

`Map` represents:

Key → Value

A → 100

B → 200

C → 300

A Map has **keys and values**, whereas a Collection represents a group of elements.

Therefore:

Collection

↓

elements

Map

↓

key-value mappings

However, Map provides collection views:

```
map.keySet();  
map.values();  
map.entrySet();
```

These return collection-like views.

8. What is `Iterable` and how is it related to Collections?

`Iterable` is an interface that allows an object to be **iterated using the enhanced for loop**.

```
for (String name : list) {  
    System.out.println(name);  
}
```

`Collection` extends `Iterable`.

```
Iterable
  ↓
Collection
  ↓
List / Set / Queue
```

`Iterable` provides:

```
Iterator<T> iterator();
```

So collections can provide an iterator for traversing their elements.

9. Iterator vs ListIterator

Feature	Iterator	ListIterator
Works with	Most Collections	List only
Direction	Forward	Forward + backward
<code>remove()</code>	Yes	Yes
<code>add()</code>	No	Yes
<code>set()</code>	No	Yes
Index information	No	Yes

Example:

```
Iterator<String> it = list.iterator();
```

Forward traversal:

```
while (it.hasNext()) {
    System.out.println(it.next());
}
```

`ListIterator` :

```
ListIterator<String> it = list.listIterator();
```

It supports:

```
it.next();
it.previous();
it.add("Java");
it.set("Spring");
it.remove();
```

Interview point

`Iterator` is a general-purpose forward iterator, while `ListIterator` is specifically designed for Lists and supports bidirectional traversal and modification operations.

10. Fail-fast vs Fail-safe iteration

Fail-fast

A fail-fast iterator detects structural modification of the collection during iteration and generally throws:

```
ConcurrentModificationException
```

Example:

```
List<String> list = new ArrayList<>();

for (String s : list) {
    list.add("Java"); // structural modification
}
```

This can result in:

```
ConcurrentModificationException
```

Common examples:

```
ArrayList
HashMap
HashSet
```

Fail-safe

"Fail-safe" is a commonly used interview term, although it is **not an official Java Collections Framework classification**.

It generally refers to iterators that can tolerate concurrent modification without throwing `ConcurrentModificationException`, often by using a snapshot or weakly consistent view.

Examples:

```
CopyOnWriteArrayList → snapshot-style iteration
ConcurrentHashMap    → weakly consistent iteration
```

Important distinction

For `CopyOnWriteArrayList`:

Iterator sees a snapshot

For `ConcurrentHashMap`:

Iterator is weakly consistent
→ may reflect some concurrent changes
→ does not provide a snapshot

★ 5-Year Interview Revision

Collection

↓

List → duplicates + ordered

Set → unique elements

Queue → processing/order

Map

↓

key → value

separate hierarchy

Iterable

↓

allows iteration / enhanced for-loop

Iterator

↓

forward traversal

ListIterator

↓

forward + backward + List modifications

Fail-fast

↓

ConcurrentModificationException may occur

Concurrent collections

↓

weakly consistent / snapshot-style iteration

↓

depends on implementation

Most important interview lines

Collection represents a group of elements, while Map represents key-value associations.

List is chosen for ordered/duplicate data, Set for uniqueness, and Map for key-based lookup.

Fail-fast iterators detect unexpected structural modification and generally throw `ConcurrentModificationException` ; concurrent collection iterators may instead provide snapshot or weakly consistent behavior.



How does ArrayList work internally?
How is ArrayList different from an array?
What happens internally when you call add() on an ArrayList?
What happens when an ArrayList reaches its capacity?
How does ArrayList resize itself?
What is the default capacity of ArrayList?
Why is ArrayList's growth strategy important for performance?
What is the time complexity of add(), get(), remove() and contains() in ArrayList?
Why is random access fast in ArrayList?
Why is insertion/removal in the middle of ArrayList expensive?



ArrayList Internals — 5-Year Interview Notes

1. How does `ArrayList` work internally?

`ArrayList` is a **resizable array implementation** of the `List` interface.

Internally, it maintains an array:

```
Object[] elementData;
```

Conceptually:

```
ArrayList
  ↓
Object[] array

[ A ][ B ][ C ][ D ][ _ ][ _ ]
          ↑
        size
```

It maintains:

- **Array** → stores elements
- **size** → number of elements currently stored
- **capacity** → size of the internal array

2. How is ArrayList different from an array?

Feature	Array	ArrayList
Size	Fixed	Dynamic
Stores primitives	✓	✗ Directly
Stores objects	✓	✓
Resize	Manual	Automatic
<code>get()</code>	O(1)	O(1)
Add/remove methods	No built-in collection methods	Yes
Generics	✗	✓

Example:

```
int[] arr = new int[10];
```

Size cannot automatically grow beyond 10.

But:

```
List<Integer> list = new ArrayList<>();
```

can grow automatically.

3. What happens internally when you call `add()`?

Example:

```
list.add("Java");
```

Conceptually:

1. Check whether capacity is available
↓
2. If available → store element
↓
3. Increase size

Internally:

```
elementData[size] = element;
size++;
```

If capacity isn't available, ArrayList first **grows the internal array**.

4. What happens when ArrayList reaches its capacity?

Suppose:

```
capacity = 10
size = 10
```

Now:

```
list.add("Java");
```

There is no free slot.

ArrayList must grow.

```
Old array
[10 elements]
    ↓
create larger array
    ↓
copy existing elements
    ↓
insert new element
```

5. How does ArrayList resize itself?

Modern OpenJDK implementations generally grow the backing array by roughly **1.5x**.

Conceptually:

```
Old capacity = 10

New capacity ≈ 15
```

Then:

```
Old array
[A][B][C][D][E]
    ↓
```

```
copy
  ↓
New array
[A][B][C][D][E][_][_]
```

The actual implementation also accounts for minimum growth and maximum array-size limits.

Interview point

ArrayList doesn't resize the existing array in place. It allocates a larger array and copies the existing elements into it.

6. What is the default capacity of ArrayList?

This is a common interview trap.

For:

```
new ArrayList<>()
```

the modern implementation uses a **lazy allocation strategy**.

The backing array is initially empty and is allocated when the first element is added.

When it first needs to grow, the default capacity is generally **10**.

So don't simply say:

"ArrayList always starts with capacity 10."

Better:

An empty ArrayList doesn't immediately allocate a 10-element backing array in modern Java. Its backing array is allocated lazily, and the first growth typically creates capacity 10.

7. Why is ArrayList's growth strategy important for performance?

Because resizing requires:

```
Allocate new array
  ↓
Copy existing elements
```

↓

Replace old array

This is expensive compared with normal `add()`.

However, because capacity grows geometrically, resizing doesn't happen on every insertion.

Therefore:

Normal add $\rightarrow O(1)$ amortized

Occasional resize $\rightarrow O(n)$

Practical optimization

If you know approximately how many elements you'll store:

```
List<Employee> employees = new ArrayList<>(10000);
```

This can reduce unnecessary resizing and copying.

8. Time complexity of ArrayList operations

Operation	Complexity
<code>get(index)</code>	$O(1)$
<code>set(index)</code>	$O(1)$
<code>add(element)</code> at end	$O(1)$ amortized
<code>add(index, element)</code>	$O(n)$
<code>remove(index)</code>	$O(n)$
<code>remove(Object)</code>	$O(n)$
<code>contains()</code>	$O(n)$
<code>indexOf()</code>	$O(n)$

Important

A resize can make a particular `add()` operation **$O(n)$** , but over many insertions, append is **$O(1)$ amortized**.

9. Why is random access fast in ArrayList?

Because ArrayList uses an **array internally**.

Arrays provide direct index-based access.

```
list.get(500);
```

Conceptually:

$$\text{Base address} + \text{index} \times \text{element size}$$

↓

direct access

So:

```
list.get(index)
```

is:

$$O(1)$$

There is no need to traverse previous elements.

10. Why is insertion/removal in the middle expensive?

Because ArrayList stores elements **contiguously**.

Suppose:

```
[A][B][C][D][E]
```

Insert **X** at index **2**:

```
[A][B][X][C][D][E]
```

Elements **C**, **D**, and **E** must be shifted to the right.

Similarly, removing **C**:

```
[A][B][C][D][E]
```

becomes:

```
[A][B][D][E]
```

D and E must shift left.

Therefore:

Middle insertion/removal $\rightarrow O(n)$

★ 5-Year Interview Revision

ArrayList

↓

Resizable array

↓

Fast random access

↓

get() $\rightarrow O(1)$

add(end)

↓

$O(1)$ amortized

Capacity full

↓

Create larger array

↓

Copy elements

↓

Insert

Modern growth

↓

$\sim 1.5 \times$ capacity

Middle insertion/removal

↓

Shift elements

↓

$O(n)$

contains()

↓

Linear search

↓

$O(n)$

Most important interview answer

ArrayList is backed by a resizable array. It provides $O(1)$ random access because elements are stored in contiguous indexed positions. Appending is $O(1)$ amortized, but when capacity is exhausted it allocates a larger array and copies elements. Insertion or removal in the middle is $O(n)$ because subsequent elements have to be shifted.



How does ArrayList store objects internally?
Can ArrayList contain null?
Can ArrayList contain duplicate values?
Is ArrayList thread-safe?
How would you make an ArrayList thread-safe?
ArrayList vs synchronizedList?
ArrayList vs CopyOnWriteArrayList?



ArrayList — Advanced Interview Notes

1. How does ArrayList store objects internally?

ArrayList uses an internal `Object[]` array.

```
List<Employee> list = new ArrayList<>();
```

Conceptually:

```
ArrayList
  ↓
Object[] elementData

[ ref1 ][ ref2 ][ ref3 ][ null ][ null ]
  ↓     ↓     ↓
Obj1   Obj2  Obj3
```

The array stores **references to objects**, not the actual object data.

The actual objects are generally on the **heap**.

2. Can ArrayList contain `null`?

Yes.

```
List<String> list = new ArrayList<>();

list.add(null);
list.add("Java");
list.add(null);
```

Both `null` values are allowed.

3. Can ArrayList contain duplicate values?

Yes.

`List` allows duplicates.

```
list.add("Java");  
list.add("Java");
```

Result:

```
[Java, Java]
```

4. Is ArrayList thread-safe?

No.

`ArrayList` is not designed for concurrent modification by multiple threads.

If multiple threads modify the same `ArrayList` without proper synchronization, you can get:

- Race conditions
- Lost updates
- Inconsistent state
- Visibility problems

5. How would you make an ArrayList thread-safe?

There are several approaches.

Option 1 — `Collections.synchronizedList()`

```
List<String> list =  
    Collections.synchronizedList(new ArrayList<>());
```

Individual operations are synchronized.

Option 2 — `CopyOnWriteArrayList`

```
List<String> list = new CopyOnWriteArrayList<>();
```

Good when:

Many reads
Few writes

Option 3 — Explicit synchronization

```
synchronized (list) {  
    list.add("Java");  
}
```

Interview point

The correct approach depends on the workload. I would not automatically synchronize every ArrayList; for read-heavy concurrent workloads, `CopyOnWriteArrayList` can be appropriate, while for general synchronization a synchronized wrapper may be sufficient.

6. ArrayList vs `Collections.synchronizedList()`

Feature	ArrayList	synchronizedList
Thread-safe	✗	✓
Internal structure	ArrayList	ArrayList wrapper
Synchronization	None	Synchronizes operations
Performance	Better single-threaded	Synchronization overhead
Concurrent reads/writes	Not safe	Operations synchronized

Example:

```
List<String> list =  
    Collections.synchronizedList(new ArrayList<>());
```

Important iteration point

Even with `synchronizedList`, iteration should be externally synchronized:

```
synchronized (list) {  
    for (String value : list) {  
        System.out.println(value);  
    }  
}
```

```
}  
}
```

because the iteration consists of multiple operations.

7. ArrayList vs CopyOnWriteArrayList

Feature	ArrayList	CopyOnWriteArrayList
Thread-safe	✗	✓
Reads	Fast	Fast
Writes	Fast	Expensive
Write behavior	Modify existing array	Creates a new array
Iterator	Fail-fast behavior	Snapshot-style
Best use case	Normal applications	Read-heavy concurrent applications
Memory usage	Lower	Higher during writes

How CopyOnWriteArrayList works

Suppose:

```
[A][B][C]
```

You execute:

```
list.add("D");
```

Instead of modifying the existing array directly, it creates a new array:

```
[A][B][C][D]
```

and publishes the new array.

Therefore, readers can continue using the old snapshot while the write occurs.

Example use case

Good for data that:

is read very frequently
and modified very rarely

Examples:

- Listener lists
- Configuration snapshots
- Read-mostly application data

★ Quick Interview Revision

ArrayList
↓
Object[] internally
↓
Stores object references
↓
Duplicates allowed
↓
null allowed
↓
Not thread-safe

Thread-safe alternatives

ArrayList
├─ Collections.synchronizedList()
│ → synchronized operations
└─ CopyOnWriteArrayList
 → excellent for read-heavy workloads
 → writes are expensive

Most important interview distinction

`synchronizedList` **protects ArrayList operations using synchronization, whereas**
`CopyOnWriteArrayList` **achieves thread-safe read-heavy access by creating a new underlying array whenever the list is modified.**



How does LinkedList work internally?
What data structure does LinkedList use?
Why is LinkedList called a doubly linked list?
What is the time complexity of get/add/remove in LinkedList?
Why is get(index) slow in LinkedList?
When would you choose LinkedList over ArrayList?
ArrayList vs LinkedList — which one would you choose in a real application and why?
Why is LinkedList often slower than people expect even for insertions/removals?
Can LinkedList work as both List and Deque?
LinkedList vs ArrayDeque?



LinkedList — 5-Year Interview Notes

1. How does LinkedList work internally?

LinkedList is implemented as a **doubly linked list**.

Each element is stored inside a node.

Conceptually:

```
null ← Node A ⇌ Node B ⇌ Node C → null
      ↓       ↓       ↓
      A       B       C
```

Each node contains:

```
item
next
prev
```

2. What data structure does LinkedList use?

Java's LinkedList uses a **doubly linked list**.

Internally, it maintains references such as:

```
Node<E> first;
Node<E> last;
```

Each node approximately contains:

```
static class Node<E> {  
    E item;  
    Node<E> next;  
    Node<E> prev;  
}
```

3. Why is LinkedList called a doubly linked list?

Because every node has **two links**:

prev ← Node → next

For example:

```
prev      next  
  ↓        ↓  
[ A ] ⇌ [ B ] ⇌ [ C ]
```

prev allows movement backward and next allows movement forward.

4. What is the time complexity of get(), add(), and remove() ?

Operation	LinkedList
get(index)	O(n)
add(element) at end	O(1)
addFirst()	O(1)
addLast()	O(1)
removeFirst()	O(1)
removeLast()	O(1)
add(index, element)	O(n)
remove(index)	O(n)

Important nuance

Once you **already have the Node/iterator positioned at the required location**, insertion/removal itself is $O(1)$.

The $O(n)$ often comes from **finding that position**.

5. Why is `get(index)` slow in LinkedList?

Because LinkedList doesn't have direct index-based access.

For:

```
list.get(500);
```

LinkedList has to traverse nodes to reach index 500.

Java's implementation can start from whichever end is closer:

```
index near beginning → traverse from first
```

```
index near end → traverse from last
```

But it still requires traversal.

Therefore:

```
get(index) →  $O(n)$ 
```

Unlike ArrayList:

```
ArrayList → direct array indexing →  $O(1)$ 
```

6. When would you choose LinkedList over ArrayList?

In **most normal application code**, ArrayList is the better default.

LinkedList can make sense when you specifically need:

- Frequent insertion/removal at the **beginning or end**
- Deque/queue operations
- A linked-list structure specifically fits the algorithm

Example:

```
Deque<String> deque = new LinkedList<>();
```

```
deque.addFirst("A");  
deque.addLast("B");  
deque.removeFirst();
```

However, for a general-purpose queue/deque, `ArrayDeque` is often preferred.

7. ArrayList vs LinkedList — which would you choose in a real application?

Usually: `ArrayList`

Why?

- $O(1)$ random access
- Better cache locality
- Less per-element memory overhead
- Usually better iteration performance
- Efficient append operations

Choose LinkedList when:

The access pattern specifically benefits from linked-node operations, especially deque-style operations, and its trade-offs are actually useful.

Interview answer

I would choose ArrayList by default. In real applications, random access and iteration are common, and ArrayList's contiguous array storage usually gives better cache locality and lower memory overhead. I would choose LinkedList only when I have a specific need for linked-list/deque semantics rather than assuming it is faster for every insertion or deletion.

8. Why is LinkedList often slower than people expect even for insertions/removals?

This is an excellent interview question.

People often remember:

```
LinkedList insertion →  $O(1)$ 
```

But that's only true **after you already have the position/node**.

Suppose:

```
list.add(500, "Java");
```

First, LinkedList must find index 500:

```
Traverse nodes
  ↓
Find position → O(n)
  ↓
Update links → O(1)
```

Overall:

$$O(n) + O(1) = O(n)$$

Also, linked nodes have:

- Object overhead
- `next` and `prev` references
- Poorer CPU cache locality
- More pointer chasing
- More allocations

So even when the theoretical insertion/removal operation is $O(1)$, **real-world performance can be worse than ArrayList**.

9. Can LinkedList work as both List and Deque?

Yes.

Java `LinkedList` implements both:

```
List
Deque
Queue
```

So the same class can be used as:

```
List<String> list = new LinkedList<>();
```

or:

```
Deque<String> deque = new LinkedList<>();
```

Deque operations include:

```
addFirst()
addLast()
removeFirst()
removeLast()
peekFirst()
peekLast()
```

10. LinkedList vs ArrayDeque

Feature	LinkedList	ArrayDeque
Internal structure	Doubly linked list	Resizable array
Implements Deque	✓	✓
Random access	O(n)	Not designed for indexed access
Add/remove ends	O(1)	O(1) amortized
Memory overhead	Higher	Lower
Cache locality	Poorer	Better
Null elements	Allowed	✗
Typical deque performance	Usually slower	Usually faster

Real-world choice

For a normal **stack/queue/deque**, prefer:

```
Deque<String> deque = new ArrayDeque<>();
```

rather than:

```
Deque<String> deque = new LinkedList<>();
```

unless you specifically need behavior that `ArrayDeque` doesn't provide.

★ Quick Interview Revision

LinkedList

↓

Doubly linked list

↓

Node = item + next + prev

get(index)

↓

$O(n)$

add/remove at ends

↓

$O(1)$

add/remove at known node

↓

$O(1)$

add/remove at arbitrary index

↓

Finding position $O(n)$

↓

Overall $O(n)$

General application

↓

ArrayList usually preferred

Deque

↓

ArrayDeque usually preferred

Most important interview line

LinkedList's insertion/removal is $O(1)$ only when the target position/node is already known. If we have to find the position by index, traversal makes the overall operation $O(n)$. In real applications, ArrayList is usually preferred because of better cache locality, lower memory overhead, and $O(1)$ random access.



If you frequently insert elements in the middle, would you always choose LinkedList? Why or why not?
Which is better for 1 million elements — ArrayList or LinkedList?
Why can ArrayList have better cache locality than LinkedList?
Why does LinkedList consume more memory?
Why might LinkedList perform worse despite $O(1)$ insertion/removal once the node is known?
What happens when you call `list.add(0, element)` on ArrayList?
What happens when you call `list.add(0, element)` on LinkedList?



ArrayList vs LinkedList — 5-Year Interview Notes

1. If you frequently insert elements in the middle, would you always choose LinkedList?

No.

This is a common misconception.

LinkedList has **$O(1)$ insertion only when the target node is already known.**

If you do:

```
list.add(500000, element);
```

LinkedList first has to traverse to that position:

```
Find node →  $O(n)$ 
    ↓
Insert node →  $O(1)$ 
    ↓
Overall →  $O(n)$ 
```

Also, LinkedList has worse cache locality and higher memory overhead.

Interview answer

I wouldn't automatically choose LinkedList just because there are frequent middle insertions. I would consider how the position is found, the access pattern,

iteration frequency, memory constraints, and actual benchmarks. If I already have an iterator/node at the insertion point, LinkedList can be advantageous. Otherwise, ArrayList may still perform better in practice.

2. Which is better for 1 million elements — ArrayList or LinkedList?

Usually **ArrayList**.

For example:

```
List<Employee> employees = new ArrayList<>(1_000_000);
```

Why?

ArrayList provides:

- Better cache locality
- Lower memory overhead
- $O(1)$ random access
- Fast iteration
- Fewer individual object allocations

LinkedList creates approximately **one node object per element**, with additional `next` and `prev` references.

Important

Don't answer:

"ArrayList is always faster."

The correct answer is:

For a typical workload with 1 million elements, I would start with ArrayList because it generally has better memory locality, lower overhead, and faster traversal. The final choice depends on the actual access and modification pattern.

3. Why can ArrayList have better cache locality than LinkedList?

ArrayList stores references in a **contiguous array**:

```
ArrayList:  
  
[ref][ref][ref][ref][ref][ref]  
  ↓   ↓   ↓   ↓  
Obj  Obj  Obj  Obj
```

The references are adjacent in memory.

When the CPU loads one part of the array into its cache, nearby references are likely to be useful soon.

LinkedList looks more like:

```
Node A           Node B           Node C  
[item|next|prev] → [item|next|prev] → [item|next|prev]
```

Nodes can be located in different areas of the heap.

So traversal involves **pointer chasing**.

Interview line

ArrayList's contiguous backing array provides better spatial locality, while LinkedList's separately allocated nodes can be scattered in memory, causing more cache misses and pointer chasing.

4. Why does LinkedList consume more memory?

ArrayList mainly stores:

```
Object[]  
  ↓  
references
```

LinkedList stores a separate Node for every element:

```
Node  
├─ item  
├─ next  
└─ prev
```

Therefore, LinkedList has overhead from:

- Node objects
- `next` reference
- `prev` reference
- Object headers/alignment
- More allocations

Conceptually:

ArrayList:

[ref][ref][ref][ref]

LinkedList:

[Node] → [Node] → [Node] → [Node]

↑ ↑
 object object
 overhead overhead

Therefore, for large collections, LinkedList can consume significantly more memory.

5. Why might LinkedList perform worse despite $O(1)$ insertion/removal once the node is known?

Because **Big-O** describes an operation under a particular assumption; it doesn't describe the entire workload or constant factors.

Suppose we already have the node:

Node A ↔ Node B ↔ Node C

Removing B is:

A.next = C
 C.prev = A

That's $O(1)$.

But LinkedList still has:

- Pointer chasing
- Poor cache locality

- More memory usage
- More object allocations
- More GC pressure

And if you need to **find** the node first:

Find node $\rightarrow O(n)$

Remove $\rightarrow O(1)$

Overall $\rightarrow O(n)$

So $O(1)$ doesn't automatically mean faster in real applications.

6. What happens when you call `list.add(0, element)` on **ArrayList**?

Suppose:

[A][B][C][D]

You call:

```
list.add(0, "X");
```

ArrayList needs to make space at index `0`.

It shifts all existing elements one position to the right:

Before:

[A][B][C][D]

After:

[X][A][B][C][D]

Conceptually:

D $\rightarrow$ index 4

C $\rightarrow$ index 3

B $\rightarrow$ index 2

```
A → index 1
X → index 0
```

Therefore:

```
add(0, element) → O(n)
```

If capacity is also full, ArrayList additionally needs to resize and copy the backing array.

7. What happens when you call `list.add(0, element)` on **LinkedList**?

Suppose:

```
A ↔ B ↔ C ↔ D
```

You call:

```
list.add(0, "X");
```

LinkedList can directly update its `first` node:

Before:

```
A ↔ B ↔ C ↔ D
```

After:

```
X ↔ A ↔ B ↔ C ↔ D
```

No shifting of all existing elements is required.

Therefore:

```
add(0, element) → O(1)
```

This is one of the cases where LinkedList has a clear algorithmic advantage over ArrayList.

★ Very Important Comparison

Operation	ArrayList	LinkedList
<code>get(index)</code>	O(1)	O(n)

Operation	ArrayList	LinkedList
<code>add(element)</code> at end	$O(1)$ amortized	$O(1)$
<code>add(0, element)</code>	$O(n)$	$O(1)$
<code>remove(0)</code>	$O(n)$	$O(1)$
Middle insertion by index	$O(n)$	$O(n)$
Remove known node	—	$O(1)$
Memory overhead	Lower	Higher
Cache locality	Better	Worse
Random access	Excellent	Poor

5-Year Interview Answer

I wouldn't choose LinkedList solely because I have frequent insertions in the middle. If I already have a reference/iterator to the insertion point, LinkedList provides $O(1)$ insertion/removal. But if I locate the position by index, traversal makes it $O(n)$. ArrayList generally performs better in real applications because of contiguous memory, better cache locality, lower memory overhead, and fast random access. For operations like `add(0, element)` specifically, LinkedList is $O(1)$, while ArrayList is $O(n)$ because elements must be shifted.



How does HashSet work internally?
How does HashSet ensure uniqueness?
Why does HashSet use HashMap internally?
What happens internally when you call set.add(element)?
How do equals() and hashCode() affect HashSet?
What happens if two objects have the same hashCode?
Can HashSet contain null?
Can HashSet contain multiple null values?
Is HashSet ordered?
HashSet vs LinkedHashSet?
HashSet vs TreeSet?
What happens if a mutable object is added to HashSet and then its fields change?



HashSet — 5-Year Interview Notes

1. How does HashSet work internally?

HashSet internally uses a **HashMap**.

Conceptually:

```
HashSet
  ↓
HashMap<E, Object>
  ↓
key = element
value = dummy Object
```

For example:

```
Set<String> set = new HashSet<>();
set.add("Java");
```

Internally, conceptually:

```
HashMap

"Java" → PRESENT
```

The actual value is only a dummy marker. The important part is the **key**.

2. How does HashSet ensure uniqueness?

HashSet uses:

```
hashCode()  
  ↓  
find bucket  
  ↓  
equals()  
  ↓  
same object/key?
```

If an equivalent element already exists, `add()` returns `false` and doesn't add another copy.

```
set.add("Java"); // true  
set.add("Java"); // false
```

3. Why does HashSet use HashMap internally?

Because HashMap already provides the exact functionality HashSet needs:

- Hashing
- Bucket management
- Collision handling
- `hashCode()`
- `equals()`
- Efficient lookup

HashSet only needs the **keys**, so it stores every set element as a HashMap key and uses a dummy value.

Interview answer

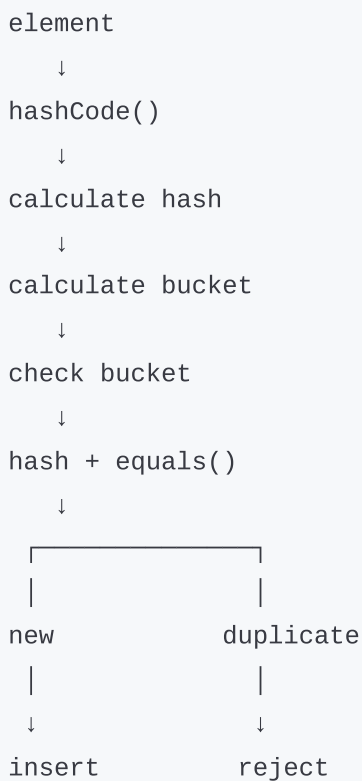
HashSet is essentially a HashMap-based implementation where the set elements are stored as keys and a constant dummy object is used as the value. This allows HashSet to reuse HashMap's hashing and collision-handling mechanism.

4. What happens internally when `set.add(element)` is called?

Example:

```
set.add(employee);
```

Conceptually:



Since HashSet uses HashMap internally, the actual insertion is essentially a `HashMap.put()` operation using the element as the key.

5. How do `equals()` and `hashCode()` affect HashSet?

Both are critical.

`hashCode()`

Used to determine the likely bucket.

`equals()`

Used to determine whether an existing element is logically equal.

```
hashCode()
  ↓
Find bucket quickly

equals()
```

↓

Confirm duplicate

Contract

If:

```
a.equals(b) == true
```

then:

```
a.hashCode() == b.hashCode()
```

must also be true.

6. What happens if two objects have the same `hashCode()` ?

That's a **hash collision**.

They may go into the same bucket.

HashSet then uses `equals()`.

Same `hashCode()`

↓

Same bucket

↓

`equals()`

↓

true

↓

false

↓

Duplicate

↓

Both retained

So:

Same `hashCode` does not mean duplicate.

The objects are duplicates only when `equals()` says they are equal.

7. Can HashSet contain `null` ?

Yes.

A `HashSet` can contain **one** `null` **element**.

```
Set<String> set = new HashSet<>();

set.add(null);
set.add(null);
```

Result:

```
set.size(); // 1
```

The second `null` is considered a duplicate.

8. Can HashSet contain multiple `null` values?

No.

Only one `null` can exist because a Set doesn't allow duplicates.

```
set.add(null); // true
set.add(null); // false
```

9. Is HashSet ordered?

No.

`HashSet` does not guarantee insertion order or sorted order.

```
Set<Integer> set = new HashSet<>();

set.add(30);
set.add(10);
set.add(20);
```

You should **not rely on the iteration order**.

If you need insertion order:

```
LinkedHashSet
```

If you need sorted order:

```
TreeSet
```

10. HashSet vs LinkedHashSet

Feature	HashSet	LinkedHashSet
Duplicate elements	✗	✗
Ordering	No guaranteed order	Insertion order
Internal structure	HashMap-based	HashMap + linked ordering
Performance	Generally slightly better	Slightly more overhead
Null	One allowed	One allowed

Use:

HashSet

→ uniqueness is important

LinkedHashSet

→ uniqueness + insertion order

11. HashSet vs TreeSet

Feature	HashSet	TreeSet
Ordering	No guaranteed order	Sorted
Internal structure	HashMap	Red-Black Tree
Average basic lookup	O(1)	O(log n)
Null	One allowed	Generally not with natural ordering
Duplicates	✗	✗
Use case	Fast uniqueness checks	Sorted unique elements

Example:

```
Set<Integer> set = new TreeSet<>();
```

```
set.add(30);
```

```
set.add(10);
```

```
set.add(20);
```

Result:

```
10
20
30
```

12. What happens if a mutable object is added to HashSet and then its fields change?

This is dangerous.

Suppose:

```
Employee e = new Employee(101);

Set<Employee> set = new HashSet<>();
set.add(e);

e.setId(200);
```

Assume `id` is used in:

```
hashCode()
equals()
```

Before mutation:

```
id = 101
  ↓
hash = X
  ↓
Bucket X
```

After mutation:

```
id = 200
  ↓
hash = Y
```

The object is still physically located according to its **old hash/bucket**, but future lookup uses the **new hash**.

So:

```
set.contains(e);
```

may return:

```
false
```

even though the object is still inside the HashSet.

Interview answer

A mutable object should generally not be used as a HashSet element when fields participating in `equals()` and `hashCode()` can change after insertion. Mutation can make the object effectively unreachable during lookup.

★ Quick Revision

HashSet



Uses HashMap internally



Element → HashMap key

Dummy Object → HashMap value

add(element)



hashCode()



bucket



equals()



duplicate?

Yes

No



reject



insert

Remember these:

HashSet

→ Unique elements

→ No guaranteed order

→ One null allowed

→ Uses hashCode() + equals()

- Hash collisions handled internally
- Mutable keys/elements are dangerous

Most important 5-year interview line

HashSet guarantees uniqueness by using a HashMap internally. The element becomes the HashMap key, `hashCode()` is used to locate the bucket, and `equals()` is used to determine whether an equivalent element already exists.



How does LinkedHashSet maintain insertion order?
LinkedHashSet vs HashSet?
LinkedHashSet vs TreeSet?
When would you use LinkedHashSet in a real application



LinkedHashSet — 5-Year Interview Notes

1. How does LinkedHashSet maintain insertion order?

`LinkedHashSet` extends the `HashSet` concept and internally uses a **LinkedHashMap**.

It maintains an additional **doubly linked list** connecting the entries in insertion order.

Conceptually:

Hash structure:

Bucket



A

B

C

Insertion-order links:

A ⇌ B ⇌ C

So it gets:

Hashing → fast lookup

Linked list → predictable insertion order

Example:

```
Set<String> set = new LinkedHashSet<>();
```

```
set.add("Java");
```

```
set.add("Spring");
```

```
set.add("AWS");
```

Iteration gives:

Java
Spring
AWS

2. LinkedHashSet vs HashSet

Feature	HashSet	LinkedHashSet
Unique elements	✓	✓
Insertion order	✗ Not guaranteed	✓ Maintained
Lookup	Average $O(1)$	Average $O(1)$
Internal structure	HashMap-based	LinkedHashMap-based
Memory	Lower	Higher
Performance	Usually slightly better	Slight overhead for ordering

Simple rule

Need uniqueness only

↓

HashSet

Need uniqueness + insertion order

↓

LinkedHashSet

3. LinkedHashSet vs TreeSet

Feature	LinkedHashSet	TreeSet
Uniqueness	✓	✓
Ordering	Insertion order	Sorted order
Internal structure	Hash + linked ordering	Red-Black Tree
Basic lookup	Average $O(1)$	$O(\log n)$
Ordering based on	Insertion	Natural ordering / Comparator

Feature	LinkedHashSet	TreeSet
Memory	Moderate	Tree-node overhead

Example:

LinkedHashSet

```
set.add(30);  
set.add(10);  
set.add(20);
```

Output:

```
30  
10  
20
```

TreeSet

```
set.add(30);  
set.add(10);  
set.add(20);
```

Output:

```
10  
20  
30
```

Simple rule

Insertion order → LinkedHashSet

Sorted order → TreeSet

No ordering requirement → HashSet

4. When would you use LinkedHashSet in a real application?

Use it when you need **both uniqueness and predictable insertion order**.

Example 1 — Remove duplicates while preserving order

Suppose:

```
List<String> names =  
    List.of("Java", "Spring", "Java", "AWS", "Spring");
```

You want:

```
Java  
Spring  
AWS
```

Use:

```
Set<String> uniqueNames =  
    new LinkedHashSet<>(names);
```

You get uniqueness **without losing the original order**.

Example 2 — Processing unique events in arrival order

Suppose events arrive:

```
LOGIN  
PAYMENT  
LOGIN  
LOGOUT  
PAYMENT
```

You want each event type only once, but in the order first seen:

```
LOGIN  
PAYMENT  
LOGOUT
```

`LinkedHashSet` fits naturally.

Example 3 — Preserve user-selected order while removing duplicates

For example, user selects:

Java → AWS → Java → Kafka → AWS

You can store them in:

```
LinkedHashSet<String>
```

Result:

```
Java → AWS → Kafka
```

★ Interview Answer

I would use `LinkedHashSet` when I need Set semantics—uniqueness—but also need predictable insertion order. It provides average $O(1)$ lookup like `HashSet` while maintaining a linked ordering of entries. The trade-off is additional memory and a small performance overhead. If I need sorted elements, I would choose `TreeSet` instead.

Quick decision

```
Need unique?
|
├─ No → List
|
└─ Yes
    |
    ├─ Don't care about order → HashSet
    |
    ├─ Preserve insertion order → LinkedHashSet
    |
    └─ Need sorted order → TreeSet
```



How does TreeSet work internally?
What data structure does TreeSet use?
How does TreeSet maintain sorted order?
Comparable vs Comparator?
What happens when you add an object to TreeSet?
Can TreeSet contain null?
What is the time complexity of TreeSet operations?
TreeSet vs HashSet?
TreeSet vs LinkedHashSet?
What happens if Comparator is inconsistent with equals()?



TreeSet — 5-Year Interview Notes

1. How does TreeSet work internally?

`TreeSet` is backed by a **TreeMap**.

Conceptually:

```
TreeSet
  ↓
TreeMap
  ↓
Red-Black Tree
```

The elements of the `TreeSet` become the keys of the underlying `TreeMap`.

TreeSet therefore provides:

- Unique elements
- Sorted order
- $O(\log n)$ basic operations

2. What data structure does TreeSet use?

A **Red-Black Tree**, which is a self-balancing binary search tree.

Conceptually:



```
    /  \    /  \
   5   15 25  40
```

The tree stays approximately balanced, so operations remain:

$O(\log n)$

3. How does TreeSet maintain sorted order?

When an element is added, TreeSet compares it with existing elements using either:

Comparable

or:

Comparator

Example:

```
Set<Integer> set = new TreeSet<>();

set.add(30);
set.add(10);
set.add(20);
```

TreeSet organizes them as:

```
10
20
30
```

The ordering is maintained by the Red-Black Tree.

4. Comparable vs Comparator

Comparable

The class itself defines its **natural ordering**.

```
class Employee implements Comparable<Employee> {

    @Override
    public int compareTo(Employee other) {
        return this.id - other.id;
    }
}
```

```
}  
}
```

Then:

```
TreeSet<Employee> employees = new TreeSet<>();
```

uses `compareTo()`.

Comparator

Ordering is defined **outside the class**.

```
Comparator<Employee> byName =  
    Comparator.comparing(Employee::getName);  
  
TreeSet<Employee> employees =  
    new TreeSet<>(byName);
```

Quick difference

Comparable	Comparator
<code>compareTo()</code>	<code>compare()</code>
Defines natural ordering	Defines custom ordering
Implemented by the class	Separate object/class
Usually one natural ordering	Can have multiple orderings

5. What happens when you add an object to TreeSet?

Suppose:

```
treeSet.add(employee);
```

Conceptually:

```
Employee  
  ↓  
Compare with root  
  ↓  
smaller? → left  
larger?  → right
```

```
↓
continue comparison
↓
duplicate according to comparison?
↓
Yes → don't insert
No  → insert
↓
Rebalance Red-Black Tree
```

The important difference from HashSet is:

```
HashSet
→ hashCode() + equals()

TreeSet
→ compareTo() / Comparator
```

6. Can TreeSet contain null?

With **natural ordering**, you should consider `null` unsupported.

For example:

```
TreeSet<String> set = new TreeSet<>();

set.add(null);
```

This generally results in:

```
NullPointerException
```

because TreeSet needs to compare elements.

With a Comparator that explicitly handles nulls, null handling can be defined, e.g. using `Comparator.nullsFirst(...)`.

Interview answer

TreeSet generally does not support null with natural ordering because it needs to compare elements. If a Comparator explicitly defines how null should be ordered, null can be handled.

7. What is the time complexity of TreeSet operations?

Because TreeSet uses a balanced Red-Black Tree:

Operation	Complexity
<code>add()</code>	$O(\log n)$
<code>remove()</code>	$O(\log n)$
<code>contains()</code>	$O(\log n)$
<code>first()</code>	$O(\log n)$ in general traversal terms
<code>last()</code>	$O(\log n)$ in general traversal terms

The main interview point is:

Basic search, insertion and deletion operations are $O(\log n)$.

8. TreeSet vs HashSet

Feature	HashSet	TreeSet
Ordering	No guaranteed order	Sorted
Internal structure	HashMap	TreeMap / Red-Black Tree
<code>add()</code> average	$O(1)$	$O(\log n)$
<code>contains()</code> average	$O(1)$	$O(\log n)$
Null	One allowed	Generally no natural-order null
Best for	Fast uniqueness	Sorted uniqueness

Choose:

Need unique + fastest lookup

↓

HashSet

Need unique + sorted

↓

TreeSet

9. TreeSet vs LinkedHashSet

Feature	TreeSet	LinkedHashSet
Unique	✓	✓
Ordering	Sorted	Insertion order
Internal structure	Red-Black Tree	HashMap + linked ordering
Basic operations	$O(\log n)$	$O(1)$ average
Use case	Sorted elements	Preserve insertion order

Example:

```
TreeSet:  
30, 10, 20  
↓  
10, 20, 30
```

```
LinkedHashSet:  
30, 10, 20  
↓  
30, 10, 20
```

10. What happens if Comparator is inconsistent with equals() ?

This is **very important for TreeSet**.

Suppose:

```
Comparator<Employee> comparator =  
    Comparator.comparing(Employee::getAge);
```

Two employees:

```
Employee A → age 30, id 101  
Employee B → age 30, id 102
```

Comparator returns:

```
compare(A, B) == 0
```

even though:

```
A.equals(B) == false
```

TreeSet considers them **the same for set ordering purposes**.

So the second employee may not be added.

```
TreeSet<Employee> set = new TreeSet<>(comparator);

set.add(A); // true
set.add(B); // false
```

Why?

For TreeSet:

```
compare(a, b) == 0
    ↓
TreeSet considers them equivalent
    ↓
Doesn't add the second element
```

This can surprise developers because:

```
equals() → says different
Comparator → says same
```

Interview answer

TreeSet uses `compareTo()` or `Comparator` to determine element equivalence for ordering. If the comparison returns 0 for two objects while `equals()` returns false, TreeSet can treat those objects as duplicates and reject one of them. Therefore, ordering should ideally be consistent with equals when using TreeSet.

★ Quick 5-Year Revision

```
TreeSet
    ↓
TreeMap
    ↓
Red-Black Tree
```

↓
Sorted + Unique
↓
 $O(\log n)$

Comparison mechanism:

HashSet
→ hashCode() + equals()

TreeSet
→ compareTo() / Comparator

Ordering:

HashSet
→ No guaranteed order

LinkedHashSet
→ Insertion order

TreeSet
→ Sorted order

Most important interview line

TreeSet is a sorted Set backed by a Red-Black Tree through TreeMap. It uses Comparable or Comparator for ordering, and comparison returning 0 means the elements are considered equivalent by the TreeSet, regardless of what equals() says.



What changed in HashMap after Java 8?
Why does HashMap use a Red-Black Tree?
What is treeification?
What happens if a mutable object is used as a key?
Why can HashMap have one null key?
Can HashMap have multiple null values?
Why is HashMap not thread-safe



HashMap — Java 8+ Interview Notes

1. What changed in HashMap after Java 8?

The major change was **collision handling**.

Before Java 8

A bucket mainly used a **linked list**:

```
Bucket
  ↓
Node → Node → Node → Node
```

If many keys collided, lookup could degrade toward:

$O(n)$

Java 8+

When a bucket becomes sufficiently large, HashMap can convert the linked list into a **Red-Black Tree**:

```
Bucket
  ↓
Red-Black Tree
    20
   /  \
  10   30
```

This improves lookup in a heavily-collided bucket to approximately:

$O(\log n)$

Other important Java 8 changes include the use of **CAS in some insertion paths** and revised resizing/tree-bin implementation details.

2. Why does HashMap use a Red-Black Tree?

To protect performance from **large numbers of hash collisions**.

Without treeification:

```
Many collisions
  ↓
Long linked list
  ↓
O(n) lookup
```

With treeification:

```
Many collisions
  ↓
Red-Black Tree
  ↓
O(log n) lookup
```

So the main reason is:

To prevent a heavily-collided bucket from behaving like a long linear linked list.

3. What is treeification?

Treeification means converting a heavily populated bucket's linked-list structure into a **Red-Black Tree**.

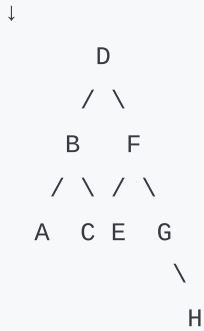
For example:

```
Before:

Bucket
  ↓
A → B → C → D → E → F → G → H
```

After treeification:

Bucket



Important thresholds

```
TREEIFY_THRESHOLD = 8
```

When a bucket reaches the relevant collision threshold, HashMap considers treeification.

But there is an important condition:

```
MIN_TREEIFY_CAPACITY = 64
```

If the table is smaller than 64, HashMap generally **resizes instead of treeifying**.

Why?

A small table with many collisions may simply need more buckets rather than a tree.

4. What happens if a mutable object is used as a HashMap key?

It can cause serious problems.

Suppose:

```
Employee e = new Employee(101);

Map<Employee, String> map = new HashMap<>();
map.put(e, "Manoj");
```

Assume `id` participates in:

```
equals()
hashCode()
```

Initially:

```
id = 101
  ↓
hash = X
  ↓
bucket X
```

Then:

```
e.setId(200);
```

Now:

```
id = 200
  ↓
hash = Y
```

But the object is still physically stored in the bucket determined using the **old hash**.

So:

```
map.get(e);
```

may fail to find the entry.

Interview answer

A key should ideally be immutable with respect to the fields used by `equals()` and `hashCode()`. If those fields change after insertion, the key may become unreachable through normal HashMap lookup.

5. Why can HashMap have one null key?

HashMap allows **one** `null` **key**.

```
map.put(null, "A");
map.put(null, "B");
```

The second operation doesn't create another null-key entry.

Instead, it replaces the value:

```
null → B
```

Why only one?

Because a Map cannot have duplicate keys.

Conceptually:

Key → Value

null → B

There is only one `null` key.

6. Can HashMap have multiple null values?

Yes.

Values don't need to be unique.

```
map.put("A", null);  
map.put("B", null);  
map.put("C", null);
```

This is valid:

```
A → null  
B → null  
C → null
```

So remember:

HashMap

```
null keys    → 1  
null values  → multiple
```

7. Why is HashMap not thread-safe?

Because HashMap doesn't provide synchronization for concurrent modifications.

Suppose:

```
Thread 1 → put(A)  
Thread 2 → put(B)  
Thread 3 → remove(A)
```

Without synchronization, operations can interfere with each other.

Possible problems include:

- Race conditions
- Lost updates
- Inconsistent state
- Visibility issues
- Unsafe concurrent iteration
- Unexpected results during resizing/modification

Therefore, if multiple threads need concurrent access and modification, use an appropriate concurrent design, commonly:

```
ConcurrentHashMap<K, V> map = new ConcurrentHashMap<>();
```

Interview answer

HashMap is not thread-safe because its operations are not synchronized and it provides no guarantees for concurrent modifications. If multiple threads modify the same map, race conditions and inconsistent behavior can occur. For concurrent access, I would generally consider ConcurrentHashMap.

★ Quick 5-Year Revision

```
Java 8 HashMap
    ↓
Collision handling improved
    ↓
Linked List
    ↓
Heavy collision
    ↓
Red-Black Tree
    ↓
O(log n)
```

```
TREEIFY_THRESHOLD = 8
MIN_TREEIFY_CAPACITY = 64
```

```
Mutable key
    ↓
hashCode changes
```

↓
Stored in old bucket
↓
get() may fail

HashMap
→ 1 null key
→ Multiple null values
→ Not thread-safe

★ Interview-ready summary

The major Java 8 HashMap improvement was converting heavily-collided buckets from linked lists into Red-Black Trees, improving worst-case bucket operations from $O(n)$ toward $O(\log n)$. Treeification occurs around the collision threshold, but HashMap first considers resizing when the table is small. Keys should remain stable with respect to `equals()` and `hashCode()`, and HashMap supports one null key and multiple null values but is not thread-safe.



How does LinkedHashMap work internally?
HashMap vs LinkedHashMap?
How does LinkedHashMap maintain insertion order?
What is access-order in LinkedHashMap?
How can LinkedHashMap be used to implement an LRU cache?
Why is LinkedHashMap useful for caching



LinkedHashMap — 5-Year Interview Notes

1. How does LinkedHashMap work internally?

`LinkedHashMap` extends the behavior of `HashMap`.

Internally, it uses:

Hash table
+
Doubly linked list

Conceptually:

HashMap buckets:

Bucket
↓
Entry A
Entry B
Entry C

Linked order:

A ⇄ B ⇄ C

Each entry maintains links to the previous and next entry, allowing `LinkedHashMap` to maintain a predictable iteration order.

2. HashMap vs LinkedHashMap

Feature	HashMap	LinkedHashMap
Hash-based lookup	✓	✓
Ordering	No guaranteed order	Predictable order
Default order	—	Insertion order
Access-order option	✗	✓
Average <code>get()</code>	$O(1)$	$O(1)$
Memory	Lower	Higher
Performance	Usually slightly faster	Slight overhead

Simple rule

```

HashMap
→ Fast lookup
→ Order doesn't matter

LinkedHashMap
→ Fast lookup
→ Need predictable ordering

```

3. How does LinkedHashMap maintain insertion order?

When a new entry is inserted, LinkedHashMap adds it to the **doubly linked list**.

Example:

```

map.put("A", 1);
map.put("B", 2);
map.put("C", 3);

```

The linked structure is approximately:

```

head
↓
A ⇌ B ⇌ C
      ↑
    tail

```

Therefore iteration gives:

A
B
C

The hash table is still used for efficient lookup; the linked list is responsible for the iteration order.

4. What is access-order in LinkedHashMap?

By default, LinkedHashMap maintains **insertion order**.

But you can create it in **access-order** mode:

```
Map<String, Integer> map =  
    new LinkedHashMap<>(16, 0.75f, true);
```

The third parameter:

```
true → access-order  
false → insertion-order
```

In access-order mode, accessing an existing entry moves it toward the **end** of the linked list.

Example:

Initial:

A → B → C

Call:

```
map.get("A");
```

Now:

B → C → A

So the order represents **least recently accessed → most recently accessed**.

This is particularly useful for implementing an LRU cache.

5. How can LinkedHashMap be used to implement an LRU cache?

LRU = Least Recently Used.

The idea is:

Recently used → keep
Least recently used → remove

Create LinkedHashMap using access-order:

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {  
  
    private final int capacity;  
  
    public LRUCache(int capacity) {  
        super(capacity, 0.75f, true);  
        this.capacity = capacity;  
    }  
  
    @Override  
    protected boolean removeEldestEntry(  
        Map.Entry<K, V> eldest) {  
        return size() > capacity;  
    }  
}
```

Usage:

```
LRUCache<Integer, String> cache =  
    new LRUCache<>(3);  
  
cache.put(1, "A");  
cache.put(2, "B");  
cache.put(3, "C");
```

If:

```
cache.get(1);
```

then **1** becomes recently used.

If you subsequently add:

```
cache.put(4, "D");
```

the least recently used entry can be automatically removed through:

```
removeEldestEntry()
```

Conceptually

Before:

```
2 → 3 → 1
↑       ↑
LRU     MRU
```

Add 4

```
3 → 1 → 4
↑       ↑
LRU     MRU
```

6. Why is LinkedHashMap useful for caching?

Because it combines:

```
HashMap
  ↓
Fast key lookup ~ O(1)

+

Linked ordering
  ↓
Track insertion/access order
```

For an LRU cache, we need both:

1. Find value quickly by key
2. Know which entry was least recently used

LinkedHashMap provides both mechanisms conveniently.

Interview answer

LinkedHashMap is useful for LRU-style caching because it provides hash-based O(1) average lookup while maintaining access order. With access-order enabled

and `removeEldestEntry()` **overridden, the least recently used entry can be automatically evicted when the cache exceeds its capacity.**

★ Quick Revision

LinkedHashMap
↓
HashMap functionality
+
Doubly linked ordering

Two modes:

Insertion order:

A → B → C

Access order:

A → B → C

get(A)

B → C → A

LRU:

Access-order LinkedHashMap
↓
Recently used → end
Least recently used → beginning
↓
`removeEldestEntry()`
↓
Eviction

★ Most important interview line

LinkedHashMap maintains a doubly linked ordering of its entries on top of hash-table storage. In insertion-order mode it preserves insertion order; in access-

order mode, accesses move entries to the end, making it a simple building block for an LRU cache.



What is Queue?
What is the difference between Queue and Deque?
What is ArrayDeque?
ArrayDeque vs LinkedList?
Why is ArrayDeque generally preferred over LinkedList for queue/deque usage?
Can ArrayDeque contain null?
What is PriorityQueue?
How does PriorityQueue work internally?
What data structure does PriorityQueue use?
PriorityQueue vs TreeSet?
How does PriorityQueue determine which element comes first?
What happens if you iterate over PriorityQueue?
Is PriorityQueue thread-safe?



Queue, Deque & PriorityQueue — 5-Year Interview Notes

1. What is Queue?

`Queue` represents a collection designed mainly for **processing elements in a particular order**.

The common model is **FIFO**:

First In → First Out

```
A → B → C
↑       ↓
first  last
```

Common methods:

```
offer() // add
poll()  // remove
peek()  // view head
```

Example:

```
Queue<String> queue = new ArrayDeque<>();
```

```
queue.offer("A");
queue.offer("B");

queue.poll(); // A
```

2. Queue vs Deque

`Deque` means **Double Ended Queue**.

A normal Queue generally focuses on one end for insertion and the other for removal.

A Deque allows operations from **both ends**.

Queue:

add → [A B C] → remove

Deque:

addFirst ← [A B C] → addLast
removeFirst ← → removeLast

Feature	Queue	Deque
Insert	Usually at tail	Both ends
Remove	Usually from head	Both ends
FIFO	✓	Can implement
LIFO/Stack	Not typical	✓

3. What is ArrayDeque?

`ArrayDeque` is a **resizable-array implementation of the Deque interface**.

Conceptually:

ArrayDeque
↓
Resizable circular array

It can efficiently perform operations at both ends:

```
addFirst()
addLast()
removeFirst()
removeLast()
peekFirst()
peekLast()
```

It can therefore be used as:

```
Queue
Stack
Deque
```

4. ArrayDeque vs LinkedList

Feature	ArrayDeque	LinkedList
Internal structure	Resizable array	Doubly linked list
Implements Deque	✓	✓
Add/remove at ends	$O(1)$ amortized	$O(1)$
Memory overhead	Lower	Higher
Cache locality	Better	Worse
Null elements	✗	✓
Typical deque performance	Better	Usually slower

5. Why is ArrayDeque generally preferred over LinkedList for Queue/Deque usage?

Because ArrayDeque usually provides better real-world performance.

Reasons:

- Better cache locality
- Less object allocation
- Lower memory overhead
- No separate Node object for each element

- Efficient operations at both ends

Conceptually:

ArrayDeque:

[A][B][C][D]

versus:

LinkedList:

[A|prev|next] → [B|prev|next] → [C|prev|next]

Interview answer

For a normal Queue or Deque, I would generally prefer ArrayDeque because it uses a resizable array, has better cache locality and lower memory overhead, and provides efficient operations at both ends. LinkedList is usually unnecessary unless its specific behavior is required.

6. Can ArrayDeque contain null?

No.

```
Deque<String> deque = new ArrayDeque<>();  
  
deque.add(null);
```

throws:

NullPointerException

Why?

Because `null` is not allowed and could otherwise make it ambiguous whether an empty result means "no element" or an actual null element.

7. What is PriorityQueue?

`PriorityQueue` is a Queue where elements are processed according to **priority**, rather than simple FIFO order.

Example:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.offer(30);
pq.offer(10);
pq.offer(20);
```

Calling:

```
pq.poll();
```

returns:

```
10
```

By default, Java's PriorityQueue is a **min-heap**.

8. How does PriorityQueue work internally?

It uses a **heap**.

By default:

```
Min-Heap
```

Conceptually:

```
      10
     /  \
    20   30
   /  \
  40   50
```

The smallest element is always at the root/head.

When you call:

```
pq.poll();
```

the root is removed and the heap is reorganized to maintain the heap property.

9. What data structure does PriorityQueue use?

A **priority queue implemented using a binary heap**, specifically a **resizable array-based binary heap** in Java.

```
PriorityQueue
  ↓
Resizable Object[]
  ↓
Binary Heap
```

By default:

Min-Heap

You can create a max-heap using a Comparator:

```
PriorityQueue<Integer> pq =
    new PriorityQueue<>(Comparator.reverseOrder());
```

10. PriorityQueue vs TreeSet

Feature	PriorityQueue	TreeSet
Main purpose	Process highest/lowest priority	Maintain sorted unique elements
Internal structure	Binary heap	Red-Black Tree
Duplicates	✓	✗
<code>peek()</code>	$O(1)$	First/lowest element can be accessed efficiently
<code>offer/add()</code>	$O(\log n)$	$O(\log n)$
<code>poll/remove head</code>	$O(\log n)$	$O(\log n)$
Full sorted iteration	✗	✓

Key difference

PriorityQueue guarantees the **head has the highest priority** according to its ordering.

It does **not** guarantee that the entire collection is sorted during iteration.

TreeSet maintains a sorted set.

11. How does PriorityQueue determine which element comes first?

Using:

```
Comparable
```

or:

```
Comparator
```

By default:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

uses natural ordering.

Therefore:

```
10 < 20 < 30
```

and:

```
pq.poll()
```

returns `10` first.

For custom priority:

```
PriorityQueue<Employee> pq =  
    new PriorityQueue<>(  
        Comparator.comparing(Employee::getSalary)  
    );
```

The Comparator determines the priority ordering.

12. What happens if you iterate over PriorityQueue?

This is an important interview trap.

Iteration does NOT guarantee sorted order.

Example:

```
PriorityQueue<Integer> pq =  
    new PriorityQueue<>();
```

```
pq.add(30);
pq.add(10);
pq.add(20);
pq.add(5);
```

You should **not assume** that:

```
for (Integer i : pq)
```

will produce:

```
5 10 20 30
```

The only guaranteed priority behavior is that:

```
pq.peek()
pq.poll()
```

operate on the highest-priority element according to the queue's ordering.

If you want elements in priority order, repeatedly use:

```
while (!pq.isEmpty()) {
    System.out.println(pq.poll());
}
```

13. Is PriorityQueue thread-safe?

No.

`PriorityQueue` is not thread-safe.

If multiple threads concurrently modify it, you need external synchronization or a concurrent alternative.

For concurrent producer/consumer scenarios, consider:

```
PriorityBlockingQueue
```

from `java.util.concurrent`.

★ Quick Revision

Queue

→ Usually FIFO

Deque

→ Operations at both ends

ArrayDeque

→ Resizable array

→ Fast Queue/Deque

→ No null

PriorityQueue

→ Priority-based processing

→ Binary heap

→ Default = min-heap

→ add/offer = $O(\log n)$

→ poll = $O(\log n)$

→ peek = $O(1)$

→ Not thread-safe

★ Most important interview distinctions

ArrayDeque

→ Queue / Deque

→ FIFO or LIFO

→ Generally preferred over LinkedList

PriorityQueue

→ Priority-based processing

→ Binary heap

→ Only head is guaranteed to have highest priority

→ Iteration is NOT sorted

TreeSet

→ Sorted + unique

→ Red-Black Tree

→ Entire iteration is sorted

5-year interview answer:

For normal queue/dequeue operations, I prefer ArrayDeque because its array-based implementation gives good cache locality and low memory overhead. For priority-based processing, I use PriorityQueue, which is backed by a binary heap and provides $O(\log n)$ insertion/removal and $O(1)$ access to the highest-priority element. I don't rely on PriorityQueue's iterator for sorted output; I use repeated `poll()` when processing elements by priority.



How does Iterator work?
What is ConcurrentModificationException?
Why does this code throw ConcurrentModificationException?
`List<Integer> list = new ArrayList<>();`

`for (Integer value : list) {
 if (value == 10) {
 list.remove(value);
 }
}`
How do you safely remove elements while iterating?
Iterator.remove() vs Collection.remove()?
What is fail-fast behavior?
Is fail-fast guaranteed?
What is the difference between Iterator and ListIterator?



Iterator & ConcurrentModificationException — 5-Year Interview Notes

1. How does Iterator work?

`Iterator` provides a standard way to traverse a collection without exposing its internal structure.

Example:

```
Iterator<Integer> iterator = list.iterator();  
  
while (iterator.hasNext()) {  
    Integer value = iterator.next();  
}
```

Internally, the iterator maintains traversal state and, for many standard collections such as `ArrayList`, tracks a modification count to detect unexpected structural modifications.

Conceptually:

```
Iterator  
  ↓  
hasNext()  
  ↓  
next()
```

↓
next element

2. What is ConcurrentModificationException?

`ConcurrentModificationException` (CME) is a **runtime exception** that can occur when a collection is structurally modified while it is being iterated in an unsupported way.

Example:

```
for (Integer value : list) {  
    list.remove(value);  
}
```

The collection is modified directly while its iterator is active.

3. Why does this code throw ConcurrentModificationException?

```
List<Integer> list = new ArrayList<>();  
  
for (Integer value : list) {  
    if (value == 10) {  
        list.remove(value);  
    }  
}
```

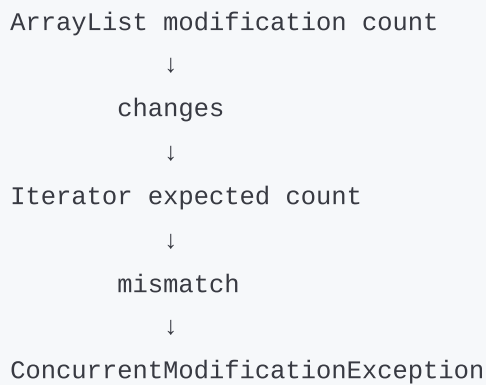
The enhanced `for` loop internally uses an `Iterator`.

Conceptually:

```
Iterator<Integer> it = list.iterator();  
  
while (it.hasNext()) {  
    Integer value = it.next();  
  
    if (value == 10) {  
        list.remove(value); // direct modification  
    }  
}
```

The iterator has its own expected modification state.

When `list.remove()` modifies the `ArrayList`:



The key issue is:

The collection was modified directly instead of through the iterator that is performing the traversal.

4. How do you safely remove elements while iterating?

Option 1 — `Iterator.remove()`

```
Iterator<Integer> iterator = list.iterator();

while (iterator.hasNext()) {
    Integer value = iterator.next();

    if (value == 10) {
        iterator.remove();
    }
}
```

This is the traditional safe approach.

Option 2 — `removeIf()`

For modern Java, this is often cleaner:

```
list.removeIf(value -> value == 10);
```

For simple filtering/removal, I would generally prefer `removeIf()`.

5. `Iterator.remove()` vs `Collection.remove()`

`Iterator.remove()`

Removes the **last element returned by that iterator**.

```
iterator.next();
iterator.remove();
```

The iterator knows about the modification and updates its internal state appropriately.

```
Collection.remove()
```

Directly modifies the collection:

```
list.remove(value);
```

If you're currently iterating using an iterator, this can invalidate the iterator's expected state and cause CME.

	Iterator.remove()	Collection.remove()
Removes	Last element returned by iterator	Matching element/object
During iteration	Safe	Can cause CME
Iterator state updated	✓	✗

6. What is fail-fast behavior?

Fail-fast means an iterator tries to detect an unexpected structural modification and fails quickly, typically by throwing `ConcurrentModificationException`.

Example:

```
Iterator created
  ↓
Collection modified externally
  ↓
Iterator detects modification
  ↓
ConcurrentModificationException
```

This is useful because it exposes incorrect concurrent/modification usage rather than allowing the iterator to continue with potentially inconsistent traversal.

7. Is fail-fast guaranteed?

No.

This is a very important interview point.

Fail-fast behavior is generally described as a **best-effort mechanism**, not a correctness guarantee.

You should **not write application logic that depends on** `ConcurrentModificationException` **being thrown**.

Interview answer

Fail-fast iterators are best-effort. Java does not guarantee that every concurrent or structural modification will always result in `ConcurrentModificationException`, particularly in the presence of unsynchronized concurrent access. They are primarily intended to detect programming errors early.

8. Iterator vs ListIterator

`ListIterator` is an enhanced iterator specifically for `List`.

Feature	Iterator	ListIterator
Works with	Collections generally	List only
Forward traversal	✓	✓
Backward traversal	✗	✓
<code>remove()</code>	✓	✓
<code>add()</code>	✗	✓
<code>set()</code>	✗	✓
Index information	✗	✓

Iterator

```
Iterator<String> it = list.iterator();
```

Can move:

A → B → C → D

ListIterator

```
ListIterator<String> it = list.listIterator();
```

Can move:

```
A → B → C → D  
← ← ← ←
```

And it supports:

```
it.add("X");  
it.remove();  
it.set("Y");
```

It also provides:

```
nextIndex();  
previousIndex();
```

★ Quick 5-Year Revision

```
Iterator  
  ↓  
Traversal mechanism  
  ↓  
hasNext()  
next()  
remove()
```

```
Direct collection modification during iteration  
  ↓  
  Iterator may detect it  
  ↓  
  ConcurrentModificationException
```

Safe removal

```
Iterator.remove()  
  OR  
Collection.removeIf()
```

Important distinction

```
Iterator.remove()
```

→ Safe removal through active iterator


```
Collection.remove()
```

→ Direct collection modification
→ Can invalidate iterator

Fail-fast

Best effort
≠
Guaranteed behavior

Iterator vs ListIterator

```
Iterator
```

→ Forward
→ remove()


```
ListIterator
```

→ Forward + backward
→ add()
→ remove()
→ set()
→ index information

★ Interview-ready answer

An Iterator maintains traversal state and, in many standard collections, detects unexpected structural modifications using a modification count. Directly modifying an ArrayList while iterating can cause ConcurrentModificationException. The safe approach is to use `Iterator.remove()` or, for simple filtering, `removeIf()`. Fail-fast behavior is best-effort rather than guaranteed. ListIterator extends Iterator functionality for Lists by supporting bidirectional traversal and `add()`, `set()`, and index operations.



Why isn't HashMap thread-safe?
What happens if multiple threads modify an ArrayList?
How can you make a List thread-safe?
Collections.synchronizedList() vs CopyOnWriteArrayList?
When would you use CopyOnWriteArrayList?
How does CopyOnWriteArrayList work internally?
Why is CopyOnWriteArrayList good for read-heavy workloads?
Why is CopyOnWriteArrayList bad for frequent writes?
HashMap vs ConcurrentHashMap?
How does ConcurrentHashMap work internally?



Collections & Concurrency — 5-Year Interview Notes

1. Why isn't HashMap thread-safe?

`HashMap` doesn't synchronize its operations.

If multiple threads modify the same `HashMap` concurrently:

```
Thread 1 → put(A)
Thread 2 → put(B)
Thread 3 → remove(A)
```

operations can interfere with each other.

Possible issues:

- Race conditions
- Lost updates
- Inconsistent state
- Visibility issues
- Unsafe iteration

For concurrent access, use `ConcurrentHashMap` when appropriate.

2. What happens if multiple threads modify an ArrayList?

`ArrayList` is also **not thread-safe**.

If multiple threads perform operations such as:

```
list.add(value);
```

simultaneously, you can get:

- Race conditions
- Lost updates
- Inconsistent state
- Visibility problems
- `ConcurrentModificationException` during iteration in some cases

For example, two threads may both observe the same current `size` and attempt conflicting updates.

Important

`ConcurrentModificationException` does **not** mean "multiple threads are always involved." A single thread can also trigger it by structurally modifying a collection directly while using its iterator.

3. How can you make a List thread-safe?

Common options:

1. `Collections.synchronizedList()`

```
List<String> list =  
    Collections.synchronizedList(new ArrayList<>());
```

Operations are synchronized.

2. `CopyOnWriteArrayList`

```
List<String> list =  
    new CopyOnWriteArrayList<>();
```

Best suited for read-heavy workloads.

3. Explicit synchronization

```
synchronized (list) {
    list.add("Java");
}
```

4. Other concurrent design

Depending on the requirement, you may use a queue or another concurrent collection instead of a List.

4. `Collections.synchronizedList()` VS `CopyOnWriteArrayList`

Feature	<code>synchronizedList</code>	<code>CopyOnWriteArrayList</code>
Thread-safe	✓	✓
Synchronization	Lock-based	Copy-on-write
Reads	Synchronized	Very efficient
Writes	Relatively cheaper	Expensive
Iteration	Requires external synchronization for compound traversal	Snapshot-style
Best for	General synchronized access	Read-heavy workloads
Memory usage	Lower	Higher during writes

Simple rule

```
General concurrent List
    ↓
synchronizedList

Many reads + very few writes
    ↓
CopyOnWriteArrayList
```

5. When would you use `CopyOnWriteArrayList`?

Use it when:

```
Reads >>> Writes
```

Examples:

- Listener/observer lists
- Configuration snapshots
- Subscriber lists
- Read-mostly application metadata

For example, if a list is read thousands of times but modified only occasionally, `CopyOnWriteArrayList` can be a good fit.

6. How does `CopyOnWriteArrayList` work internally?

The key idea is:

Readers use the current array, while a write creates a new copy.

Suppose:

Current array:

[A][B][C]

A writer executes:

```
list.add("D");
```

Conceptually:

Old array:

[A][B][C]

↓ copy

New array:

[A][B][C][D]

↓

Publish new array

Readers that were already using the old array can continue safely.

The write operation uses locking to coordinate modifications.

7. Why is CopyOnWriteArrayList good for read-heavy workloads?

Because reads don't need to acquire the same lock used for writes.

Conceptually:

```
Readers
  ↓ ↓ ↓ ↓
[A][B][C]
```

can access the current snapshot efficiently.

Meanwhile:

```
Writer
  ↓
creates new array
  ↓
publishes it
```

Therefore, when reads vastly outnumber writes, the cost of copying on occasional writes can be worthwhile.

8. Why is CopyOnWriteArrayList bad for frequent writes?

Every structural modification generally requires a **new array and copying the existing elements**.

Suppose:

```
1 million elements
```

and you repeatedly do:

```
list.add(element);
```

You can end up doing lots of copying and temporary allocations.

This causes:

- Higher CPU usage
- More memory allocation
- More GC pressure

- Poor write performance

So:

Frequent writes



CopyOnWriteArrayList ❌

9. HashMap vs ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap
Thread-safe	❌	✅
Null key	One allowed	❌
Null values	Multiple allowed	❌
Concurrent reads/writes	Not safe	Designed for it
Locking	None	Fine-grained concurrency mechanisms
Performance	Excellent single-threaded	Excellent concurrent access

Important null difference

ConcurrentHashMap doesn't allow:

```
map.put(null, value);  
map.put(key, null);
```

because `null` is used to represent absence in its APIs, and allowing null would create ambiguity in concurrent operations.

10. How does ConcurrentHashMap work internally?

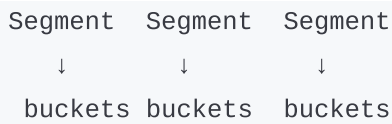
The implementation has evolved across Java versions.

Java 7

It used **segments**:

ConcurrentHashMap





Each segment had its own lock.

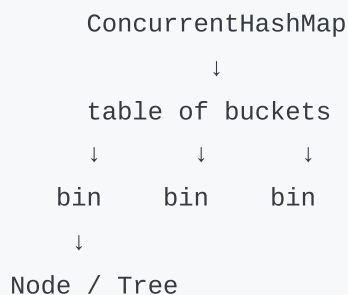
Java 8+

The segment-based design was removed.

Modern ConcurrentHashMap primarily uses:

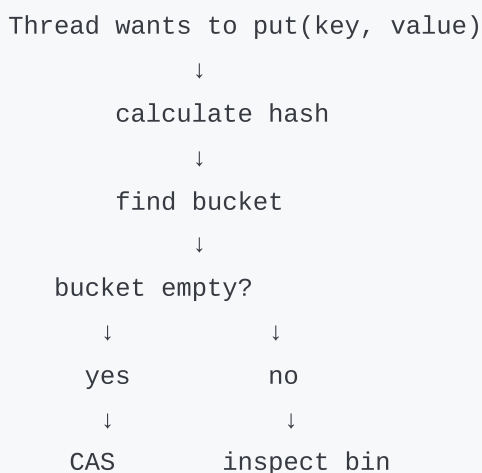
```
Hash table
+
CAS
+
synchronized on individual bins when needed
+
volatile/atomic coordination
```

Conceptually:



It allows multiple threads to operate on different parts of the map concurrently.

How does `put()` work conceptually?



↓
coordinate update
↓
insert/update node

If a bucket becomes heavily collided, the implementation can use a tree structure similar to HashMap.

★ Why ConcurrentHashMap scales better than synchronized HashMap

Consider:

```
Map<K,V> map =  
    Collections.synchronizedMap(new HashMap<>());
```

A synchronized map effectively serializes access around the map's synchronized operations.

With `ConcurrentHashMap`, concurrency is much finer-grained, allowing unrelated buckets/operations to proceed concurrently where possible.

Conceptually:

Synchronized Map:

```
Thread 1 ┌─┐  
Thread 2 ┌─┴─┐ one synchronized map  
Thread 3 ┌─┐
```

ConcurrentHashMap:

```
Thread 1 → Bucket A  
Thread 2 → Bucket B  
Thread 3 → Bucket C  
↓  
more concurrency
```

★ Quick Interview Revision

List

```
ArrayList  
→ Not thread-safe
```

synchronizedList
→ synchronized operations
→ general-purpose thread-safe wrapper

CopyOnWriteArrayList
→ copy array on writes
→ very efficient reads
→ best when reads >> writes

Map

HashMap
→ Not thread-safe
→ allows null key/value

ConcurrentHashMap
→ Thread-safe
→ doesn't allow null key/value
→ designed for concurrent access
→ CAS + fine-grained synchronization

Most important 5-year interview answer

I choose the concurrent collection based on the workload. For a general synchronized List, `Collections.synchronizedList()` is simple, while `CopyOnWriteArrayList` is better when reads vastly outnumber writes because readers can access an immutable snapshot while writers create a new array. For Maps, `ConcurrentHashMap` provides scalable concurrent access using techniques such as CAS, volatile state and fine-grained synchronization rather than synchronizing the entire map for every operation.



Why doesn't ConcurrentHashMap allow null keys/values?
ConcurrentHashMap vs Hashtable?
When would you choose ConcurrentHashMap?



ConcurrentHashMap — 5-Year Interview Notes

1. Why doesn't ConcurrentHashMap allow null keys/values?

ConcurrentHashMap does **not allow** `null` keys or values.

```
map.put(null, "Java");    // ❌  
map.put("A", null);      // ❌
```

The main reason is **ambiguity in concurrent operations**.

For example:

```
map.get("A")
```

If the result is `null`, it needs to clearly mean:

Key does not exist

If null values were allowed, `null` could mean either:

1. Key doesn't exist
2. Key exists with null value

In concurrent code, this ambiguity becomes problematic because the map can change between operations.

Interview answer

ConcurrentHashMap disallows null keys and values because `null` is used to represent absence in its concurrent APIs. Allowing null would make it ambiguous whether a null result means that the key is absent or that the key is mapped to null.

2. ConcurrentHashMap vs Hashtable

Both are thread-safe Maps, but `ConcurrentHashMap` is generally preferred for modern concurrent applications.

Feature	Hashtable	ConcurrentHashMap
Thread-safe	✓	✓
Introduced	Legacy Java	Java 5
Synchronization	Broad method-level synchronization	Fine-grained concurrency
Concurrent performance	Lower	Better
Null key	✗	✗
Null value	✗	✗
Modern choice	✗ Generally avoid	✓
Iterators	Fail-fast style	Weakly consistent

Hashtable

Historically:

Thread 1 → Hashtable



Thread 2 → waits

Its operations use synchronization that can restrict concurrency.

ConcurrentHashMap

Allows greater concurrency:

Thread 1 → Bucket A

Thread 2 → Bucket B

Thread 3 → Bucket C

Operations on unrelated portions can proceed concurrently where the implementation permits.

Interview answer

Hashtable is a legacy synchronized Map and generally provides poorer concurrency because synchronization is much broader. ConcurrentHashMap was

designed specifically for concurrent access and provides much better scalability through finer-grained coordination and non-blocking techniques such as CAS where appropriate.

3. When would you choose ConcurrentHashMap?

Choose `ConcurrentHashMap` when:

```
Multiple threads
    ↓
read/write shared Map
    ↓
need thread safety
    ↓
need good concurrency
```

Typical examples:

1. Shared application cache

```
Map<String, User> cache =
    new ConcurrentHashMap<>();
```

Multiple threads can read and update the cache safely.

2. Counters / statistics

For example, maintaining counts across multiple threads.

Often you'd combine it with atomic operations such as:

```
map.merge(key, 1, Integer::sum);
```

3. In-memory shared state

When multiple threads need to access and update shared key-value data without externally locking the entire map.

★ When NOT to choose it

Don't use `ConcurrentHashMap` just because "thread-safe is better."

If:

Only one thread accesses the Map

a normal:

HashMap

is usually sufficient.

If you need:

Sorted keys

consider:

ConcurrentSkipListMap

instead.

★ Quick Interview Decision

Single-threaded

↓

HashMap

Concurrent access

↓

ConcurrentHashMap

Legacy synchronized Map

↓

Hashtable ❌

Concurrent + sorted keys

↓

ConcurrentSkipListMap

🎯 5-Year Interview Answer

I would choose ConcurrentHashMap when multiple threads need to read and modify a shared Map concurrently and I want better scalability than synchronizing

the entire map. It doesn't allow null keys or values because null would make absence ambiguous in concurrent operations. Compared with legacy Hashtable, ConcurrentHashMap is designed for modern concurrent workloads and provides much finer-grained concurrency and better scalability.



What is the Collections class?
Difference between Collections.sort() and List.sort()?
How does Collections.unmodifiableList() work?
Difference between unmodifiable collection and immutable collection?
What is Collections.synchronizedList()?
What is Collections.emptyList()?
What is Collections.singletonList()?
What is Arrays.asList()?
Why can't we add/remove elements from Arrays.asList()?



Collections Utility Class — 5-Year Interview Notes

1. What is the `Collections` class?

`Collections` is a **utility class** in `java.util` that provides static methods for working with collections.

Examples:

```
Collections.sort(list);  
Collections.reverse(list);  
Collections.shuffle(list);  
Collections.unmodifiableList(list);  
Collections.synchronizedList(list);  
Collections.emptyList();  
Collections.singletonList(value);
```

Interview line

`Collections` **is a utility class containing static helper methods for operating on Collection types. It is different from the `Collection` interface.**

2. `Collections.sort()` VS `List.sort()`

Both can sort a List.

```
Collections.sort()
```

```
Collections.sort(list);
```

It's a utility method.

```
List.sort()
```

```
list.sort(comparator);
```

It's a **default method on the** `List` **interface**, available since Java 8.

	<code>Collections.sort()</code>	<code>List.sort()</code>
Type	Utility method	List interface method
Since	Older Java versions	Java 8
Comparator	Can be provided	Can be provided
Modern preference	Usually <code>list.sort()</code>	✅ Natural choice

Example:

```
list.sort(Comparator.comparing(Employee::getName));
```

Interview point

`List.sort()` **is the more direct modern API because sorting is an operation on a List itself.** `Collections.sort()` **remains available mainly for compatibility and convenience.**

3. How does `Collections.unmodifiableList()` work?

Example:

```
List<String> original =  
    new ArrayList<>();  
  
List<String> readOnly =  
    Collections.unmodifiableList(original);
```

The returned list is a **read-only view/wrapper**.

Trying:

```
readOnly.add("Java");
```

throws:

```
UnsupportedOperationException
```

Important

It is a **view**, not a copy.

```
original.add("Java");  
  
System.out.println(readOnly);
```

will also show:

```
[Java]
```

because both refer to the same underlying collection.

4. Unmodifiable collection vs Immutable collection

This is an important distinction.

Unmodifiable

You cannot modify the collection **through that reference**.

But the underlying collection may still change elsewhere.

```
List<String> original =  
    new ArrayList<>();  
  
List<String> view =  
    Collections.unmodifiableList(original);  
  
original.add("Java");
```

view now reflects the change.

```
Original → changes  
    ↓  
Unmodifiable view → sees change
```

Immutable

The collection's state **cannot be changed after creation**.

Example:

```
List<String> list =  
    List.of("Java", "Spring");
```

You cannot modify it, and there isn't a separate mutable backing list that you can modify to change the collection.

Interview line

An unmodifiable collection is a read-only view of another collection, while an immutable collection has state that cannot be changed after creation.

5. What is `Collections.synchronizedList()` ?

It creates a **thread-safe synchronized wrapper** around a List.

```
List<String> list =  
    Collections.synchronizedList(  
        new ArrayList<>()  
    );
```

Operations are synchronized.

Useful when you need simple thread-safe access to a List.

Important

Iteration still needs external synchronization:

```
synchronized (list) {  
    for (String value : list) {  
        System.out.println(value);  
    }  
}
```

6. What is `Collections.emptyList()` ?

Returns an **empty List**.

```
List<String> list =  
    Collections.emptyList();
```

Result:

```
[]
```

It is useful when you need to return an empty list without creating a new mutable ArrayList.

It is **unmodifiable**.

```
list.add("Java");
```

throws:

```
UnsupportedOperationException
```

7. What is `Collections.singletonList()` ?

Creates a List containing **exactly one element**.

```
List<String> list =  
    Collections.singletonList("Java");
```

Result:

```
[Java]
```

It is unmodifiable.

```
list.add("Spring"); // ❌
```

throws `UnsupportedOperationException`.

8. What is `Arrays.asList()` ?

`Arrays.asList()` creates a **fixed-size List backed by the original array**.

```
String[] arr = {"Java", "Spring"};
```

```
List<String> list =  
    Arrays.asList(arr);
```

Conceptually:

```
Array
[A][B]
  ↓  ↓
List
[A][B]
```

Changes to the array can be visible through the List and vice versa for element replacement.

9. Why can't we add/remove elements from `Arrays.asList()` ?

Because the returned List has a **fixed size**.

```
List<String> list =
    Arrays.asList("Java", "Spring");
```

This works:

```
list.set(0, "AWS");
```

because you're replacing an existing element.

But:

```
list.add("Kafka");
list.remove("Java");
```

throws:

```
UnsupportedOperationException
```

because adding/removing would change the size, which the array-backed fixed-size List cannot do.

If you need a mutable List:

```
List<String> list =
    new ArrayList<>(
        Arrays.asList("Java", "Spring")
    );
```

Now:

```
list.add("AWS");  
list.remove("Java");
```

works.

★ Quick Revision

API	Meaning
<code>Collections.sort()</code>	Utility sorting method
<code>List.sort()</code>	List's sorting method
<code>unmodifiableList()</code>	Read-only view
<code>synchronizedList()</code>	Synchronized List wrapper
<code>emptyList()</code>	Empty unmodifiable List
<code>singletonList(x)</code>	One-element unmodifiable List
<code>Arrays.asList()</code>	Fixed-size array-backed List

`Arrays.asList()` **remember:**

```
Arrays.asList()  
  ↓  
Fixed size  
  ↓  
set() → ✓  
add() → ✗  
remove() → ✗
```

★ 5-Year Interview Answer

`Collections` **is a utility class for collection operations.** `unmodifiableList()` **provides a read-only view, whereas an immutable collection cannot be changed after creation.** `synchronizedList()` **provides synchronized access to a List.** `emptyList()` **and** `singletonList()` **provide reusable unmodifiable lists.** `Arrays.asList()` **creates a fixed-size List backed by an array, so element replacement with** `set()` **is allowed, but structural operations like** `add()` **and** `remove()` **throw** `UnsupportedOperationException` .



Difference between Arrays.asList() and List.of()?
Why does List.of() not allow null?



Arrays.asList() vs List.of() — 5-Year Interview Notes

1. Arrays.asList() vs List.of()

Feature	Arrays.asList()	List.of()
Introduced	Java 1.2	Java 9
Size	Fixed-size	Immutable
set()	✅ Allowed	❌ Not allowed
add()	❌	❌
remove()	❌	❌
Null elements	✅ Allowed	❌
Backed by array	✅	❌
Changes reflected with original array	✅	Not applicable
Allows modification	Element replacement only	No modification

Example

```
String[] arr = {"Java", "Spring"};

List<String> list = Arrays.asList(arr);

list.set(0, "AWS");    // ✅
list.add("Kafka");     // ❌
```

Arrays.asList() is a **fixed-size view backed by the array**.

List.of()

```
List<String> list =
```

```
List.of("Java", "Spring");
```

It creates an **unmodifiable/immutable-style JDK collection**.

```
list.set(0, "AWS");    // ❌  
list.add("Kafka");    // ❌  
list.remove("Java");  // ❌
```

All structural and element-replacement operations are unsupported.

Simple rule

Need fixed-size List backed by an existing array

↓

`Arrays.asList()`

Need an immutable List

↓

`List.of()`

2. Why does `List.of()` not allow `null`?

`List.of()` is designed for **immutable collections**, and the JDK deliberately rejects `null`.

```
List<String> list =  
    List.of("Java", null);
```

throws:

```
NullPointerException
```

Main reasons

1. Clearer semantics

With immutable collections, the JDK APIs intentionally avoid `null` so that absence and actual values aren't mixed together.

```
null  
↓  
not a valid element
```

2. Better API consistency

The Java 9 immutable collection factories:

```
List.of()  
Set.of()  
Map.of()
```

all reject nulls.

3. Enables simpler internal implementations

Because the implementation knows:

```
No element will ever be null
```

it can avoid handling null values in its internal logic and maintain stronger invariants.

★ Interview-ready answer

`Arrays.asList()` **creates a fixed-size List backed by the original array. It allows `set()` but doesn't allow structural changes like `add()` or `remove()`, and it permits nulls.** `List.of()` **creates an unmodifiable immutable-style List introduced in Java 9. It doesn't allow any modification and rejects null elements by design.**

Quick memory trick

```
Arrays.asList()  
→ Array-backed  
→ Fixed size  
→ null allowed  
→ set() allowed
```

```
List.of()  
→ Immutable  
→ null NOT allowed  
→ set/add/remove NOT allowed
```



You need to store 10 million unique IDs and frequently check whether an ID exists. Which collection would you choose and why?

You need to preserve insertion order while preventing duplicates. Which collection?

You need sorted data with frequent lookup and insertion. Which collection?

You need key-value lookup with no ordering requirement. Which collection?

You need key-value lookup while maintaining insertion order. Which collection?



Collection Selection — 5-Year Interview Notes

1. 10 million unique IDs + frequent existence checks

Choose: `HashSet`

```
Set<Long> ids = new HashSet<>();
```

Why?

- Unique elements
- Average `contains()` → **O(1)**
- Hash-based lookup
- Suitable when ordering isn't required

For 10 million entries, I would also consider memory footprint, ID type, expected concurrency, and initial capacity when sizing the collection.

2. Preserve insertion order + prevent duplicates

Choose: `LinkedHashSet`

```
Set<String> data = new LinkedHashSet<>();
```

Why?

- Prevents duplicates
- Maintains insertion order
- Average lookup → **O(1)**

Input: A B C A D
Output: A B C D

3. Sorted data + frequent lookup and insertion

Choose: `TreeSet`

```
Set<Integer> data = new TreeSet<>();
```

Why?

- Maintains sorted order
- Unique elements
- `contains()` → **O(log n)**
- `add()` → **O(log n)**

Internally uses a **Red-Black Tree**.

If duplicates are required, I'd consider a different structure such as a sorted `List` or a map-based approach depending on the access pattern.

4. Key-value lookup + no ordering requirement

Choose: `HashMap`

```
Map<String, User> users = new HashMap<>();
```

Why?

- Average `get()` → **O(1)**
- Average `put()` → **O(1)**
- No ordering requirement
- Efficient general-purpose Map

5. Key-value lookup + insertion order

Choose: `LinkedHashMap`

```
Map<String, User> users = new LinkedHashMap<>();
```

Why?

- Average `get()` → **O(1)**
- Maintains insertion order
- Useful when you need predictable iteration order

★ Quick Decision Table

Requirement	Collection	Main reason
Unique + fast existence check	HashSet	Average O(1) lookup
Unique + insertion order	LinkedHashSet	Hashing + linked ordering
Unique + sorted	TreeSet	Red-Black Tree
Key-value + no order	HashMap	Average O(1) lookup
Key-value + insertion order	LinkedHashMap	Hashing + insertion ordering

🎯 5-Year Interview Shortcut

Unique?

- └ No → List / Map
- └ Yes
 - └ Fast lookup → HashSet
 - └ Insertion order → LinkedHashMap
 - └ Sorted → TreeSet

Key-Value?

- └ No ordering → HashMap
- └ Insertion order → LinkedHashMap

For **multi-threaded** versions, don't automatically add `synchronized`; choose based on the access pattern—for example, `ConcurrentHashMap` for concurrent key-value access.



You need an LRU cache. Which collection can help implement it?

You have 90% reads and 10% writes on a shared List. Which collection would you choose?

You have multiple threads frequently reading and modifying a Map. Which collection would you choose?

You need a queue for BFS. Which implementation would you choose?

You need a priority-based task scheduler. Which collection would you choose?



Collection Selection — Scenario-Based Interview Notes

1. You need an LRU cache. Which collection?

Choose: `LinkedHashMap`

Use **access-order** mode:

```
new LinkedHashMap<>(16, 0.75f, true);
```

Why?

- Average $O(1)$ lookup
- Maintains access order
- Recently accessed entry moves to the end
- `removeEldestEntry()` can remove the least recently used entry

LRU → B → C → A ← Most recently used

Interview answer: `LinkedHashMap` with access-order is a simple way to implement an LRU cache.

2. 90% reads + 10% writes on a shared List

Choose: `CopyOnWriteArrayList`

Why?

Reads → very frequent
Writes → relatively less frequent

`CopyOnWriteArrayList` allows efficient read access while writes create a new underlying array.

Best suited when:

Reads >>> Writes

Examples:

- Listener lists
- Configuration data
- Subscriber lists

Interview answer: Since the workload is read-heavy, I'd choose `CopyOnWriteArrayList`, accepting the higher cost of writes for efficient concurrent reads.

3. Multiple threads frequently reading and modifying a Map

Choose: `ConcurrentHashMap`

```
Map<String, User> users =  
    new ConcurrentHashMap<>();
```

Why?

- Thread-safe
- Designed for concurrent access
- Allows concurrent reads and updates
- Better scalability than synchronizing the entire Map

Interview answer: I'd use `ConcurrentHashMap` for a shared Map with frequent concurrent reads and modifications.

4. Queue for BFS

Choose: `ArrayDeque`

```
Queue<Node> queue = new ArrayDeque<>();
```

BFS requires FIFO:

First added → First processed

Operations:

```
queue.offer(node);  
queue.poll();
```

Both are efficient.

Why not LinkedList?

`LinkedList` also works as a Queue, but `ArrayDeque` is generally preferred because it has:

- Better cache locality
- Lower memory overhead
- No per-node object allocation

Interview answer: For BFS, I'd normally use `ArrayDeque` as the queue implementation.

5. Priority-based task scheduler

Choose: `PriorityQueue`

```
PriorityQueue<Task> queue =  
    new PriorityQueue<>(  
        Comparator.comparing(Task::getPriority)  
    );
```

The highest-priority task can be retrieved using:

```
queue.peek();  
queue.poll();
```

Internally:

```
PriorityQueue  
    ↓  
Binary Heap
```

Complexity:

```
peek() → O(1)
offer() → O(log n)
poll() → O(log n)
```

If multiple threads are producing/consuming tasks, consider:

```
PriorityBlockingQueue
```

instead.

Interview answer: For a single-threaded priority queue, I'd use `PriorityQueue`. For a concurrent producer-consumer scheduler, I'd consider `PriorityBlockingQueue`.

★ Quick Interview Table

Scenario	Choice	Why
LRU Cache	LinkedHashMap	Access order + O(1) average lookup
90% reads / 10% writes List	CopyOnWriteArrayList	Read-heavy workload
Concurrent Map	ConcurrentHashMap	Scalable concurrent access
BFS Queue	ArrayDeque	Efficient FIFO
Priority scheduler	PriorityQueue	Binary heap + priority ordering
Concurrent priority scheduler	PriorityBlockingQueue	Thread-safe priority queue

🎯 5-Year Shortcut

```
LRU → LinkedHashMap
Read-heavy List → CopyOnWriteArrayList
Concurrent Map → ConcurrentHashMap
BFS → ArrayDeque
Priority processing → PriorityQueue
Concurrent priority → PriorityBlockingQueue
```



You need a queue for BFS. Which implementation would you choose?

You need a priority-based task scheduler. Which collection would you choose?

You need to remove duplicates while preserving order. How would you do it?

How would you find the first non-repeating character using Java Collections?

How would you find duplicate elements in an array using Collections?

How would you count frequency of each element using a Map?

How would you sort a Map by values?

How would you find the top K frequent elements?



Java Collections — Coding-Based Interview Notes

1. Queue for BFS — which implementation?

Choose `ArrayDeque` .

```
Queue<Node> queue = new ArrayDeque<>();

queue.offer(root);

while (!queue.isEmpty()) {
    Node node = queue.poll();
    // process node
}
```

Why: BFS needs FIFO, and `ArrayDeque` provides efficient `offer()` / `poll()` with lower overhead than `LinkedList` .

2. Priority-based task scheduler — which collection?

Choose `PriorityQueue` .

```
PriorityQueue<Task> queue =
    new PriorityQueue<>()
```

```
        Comparator.comparing(Task::getPriority)
    );
```

```
peek() → O(1)
offer() → O(log n)
poll() → O(log n)
```

For multiple producer/consumer threads, use `PriorityBlockingQueue`.

3. Remove duplicates while preserving order

Use `LinkedHashSet`.

```
List<Integer> list =
    Arrays.asList(10, 20, 10, 30, 20);

List<Integer> result =
    new ArrayList<>(new LinkedHashSet<>(list));
```

Result:

```
[10, 20, 30]
```

Why: `LinkedHashSet` gives both **uniqueness + insertion order**.

4. Find the first non-repeating character

Use `LinkedHashMap` to maintain character insertion order and frequency.

```
String str = "swiss";

Map<Character, Integer> frequency =
    new LinkedHashMap<>();

for (char ch : str.toCharArray()) {
    frequency.merge(ch, 1, Integer::sum);
}

for (Map.Entry<Character, Integer> entry :
    frequency.entrySet()) {

    if (entry.getValue() == 1) {
        System.out.println(entry.getKey());
        break;
    }
}
```

```
}  
}
```

Output:

```
w
```

Why `LinkedHashMap` ?

`HashMap`
→ frequency

`LinkedHashMap`
→ frequency + original order

Time: **$O(n)$**

Space: **$O(k)$** , where `k` is the number of distinct characters.

5. Find duplicate elements in an array

A clean approach is using a `HashSet`.

```
int[] arr = {1, 2, 3, 2, 4, 1, 5};  
  
Set<Integer> seen = new HashSet<>();  
Set<Integer> duplicates = new HashSet<>();  
  
for (int num : arr) {  
    if (!seen.add(num)) {  
        duplicates.add(num);  
    }  
}  
  
System.out.println(duplicates);
```

Output:

```
[1, 2]
```

Important:

```
seen.add(num)
```

returns:

```
true → element was new  
false → element already existed
```

Time: **$O(n)$ average**

Space: **$O(n)$**

6. Count frequency of each element using a Map

Use `HashMap`.

```
int[] arr = {1, 2, 2, 3, 3, 3};  
  
Map<Integer, Integer> frequency =  
    new HashMap<>();  
  
for (int num : arr) {  
    frequency.merge(num, 1, Integer::sum);  
}
```

Result:

```
1 → 1  
2 → 2  
3 → 3
```

Alternative:

```
frequency.put(  
    num,  
    frequency.getOrDefault(num, 0) + 1  
);
```

Interview preference

`merge()` is concise and clearly expresses the frequency-counting operation.

7. Sort a Map by values

A Map itself isn't inherently sorted by values, so commonly we sort its entries.

```
Map<String, Integer> map = new HashMap<>();
```

```
map.put("A", 30);
map.put("B", 10);
map.put("C", 20);

Map<String, Integer> sorted =
    map.entrySet()
        .stream()
        .sorted(Map.Entry.comparingByValue())
        .collect(Collectors.toMap(
            Map.Entry::getKey,
            Map.Entry::getValue,
            (a, b) -> a,
            LinkedHashMap::new
        ));
```

Result:

```
B → 10
C → 20
A → 30
```

Why `LinkedHashMap` at the end?

Because after sorting the stream, we need a Map implementation that **preserves the sorted iteration order**.

For descending order:

```
.sorted(
    Map.Entry.<String, Integer>
        comparingByValue()
        .reversed()
)
```

8. Find the Top K frequent elements

Use:

```
HashMap + PriorityQueue
```

Step 1 — Count frequency

```
Map<Integer, Integer> frequency =
    new HashMap<>();

for (int num : arr) {
    frequency.merge(num, 1, Integer::sum);
}
```

Step 2 — Maintain a min-heap of size K

```
PriorityQueue<Map.Entry<Integer, Integer>> pq =
    new PriorityQueue<>(
        Map.Entry.comparingByValue()
    );

for (Map.Entry<Integer, Integer> entry :
    frequency.entrySet()) {

    pq.offer(entry);

    if (pq.size() > k) {
        pq.poll();
    }
}
```

At the end:

```
PriorityQueue
  ↓
contains K most frequent elements
```

Why a min-heap?

We want to remove the **least frequent among the current top K**.

```
Top K candidates

    min-heap
      ↓
least frequent → root
      ↓
remove when size > K
```

Complexity

For n array elements and m distinct elements:

```
Frequency counting →  $O(n)$ 
Heap processing    →  $O(m \log K)$ 

Total              →  $O(n + m \log K)$ 
```

Space:

```
 $O(m + K)$ 
```

★ 5-Year Interview Summary

Problem	Best approach
BFS	ArrayDeque
Priority tasks	PriorityQueue
Remove duplicates + preserve order	LinkedHashSet
First non-repeating character	LinkedHashMap
Find duplicates	HashSet
Frequency counting	HashMap
Sort Map by values	Stream + LinkedHashMap
Top K frequent	HashMap + PriorityQueue

🎯 Most important patterns to remember

```
Unique + order
→ LinkedHashSet

Frequency
→ HashMap

Frequency + order
→ LinkedHashMap

Top K
```

→ HashMap + Heap

BFS

→ Queue + ArrayDeque

Priority

→ PriorityQueue

These are very common **5-year Java interview coding patterns** because they test whether you can choose the appropriate collection rather than just knowing the API.



How would you implement an LRU cache?
How would you design a thread-safe in-memory cache?



1. How would you implement an LRU Cache?

For a simple LRU cache, use `LinkedHashMap` **with access-order**.

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {  
  
    private final int capacity;  
  
    public LRUCache(int capacity) {  
        super(capacity, 0.75f, true); // true = access-order  
        this.capacity = capacity;  
    }  
  
    @Override  
    protected boolean removeEldestEntry(  
        Map.Entry<K, V> eldest) {  
        return size() > capacity;  
    }  
}
```

Usage:

```
LRUCache<Integer, String> cache = new LRUCache<>(3);  
  
cache.put(1, "A");  
cache.put(2, "B");  
cache.put(3, "C");  
  
cache.get(1); // 1 becomes recently used  
cache.put(4, "D"); // least recently used entry is removed
```

Internally

```
HashMap  
↓  
Fast lookup O(1)
```

Doubly linked list

↓

Maintains access order

Least recently used → head

Most recently used → tail

5-year interview answer:

I can implement a simple LRU cache using `LinkedHashMap` in access-order mode and override `removeEldestEntry()`. This gives average $O(1)$ lookup/update and automatic eviction of the least recently used entry.

2. How would you design a thread-safe in-memory cache?

For a basic concurrent cache, I would use:

```
ConcurrentHashMap<K, V>
```

But if the requirement includes **LRU eviction, TTL, maximum size, statistics, refresh, etc.**, I wouldn't build all of that manually unless required. I'd generally use a mature caching library such as Caffeine.

Basic thread-safe cache

```
class Cache<K, V> {  
  
    private final ConcurrentHashMap<K, V> cache =  
        new ConcurrentHashMap<>();  
  
    public V get(K key) {  
        return cache.get(key);  
    }  
  
    public void put(K key, V value) {  
        cache.put(key, value);  
    }  
  
    public V remove(K key) {  
        return cache.remove(key);  
    }  
}
```

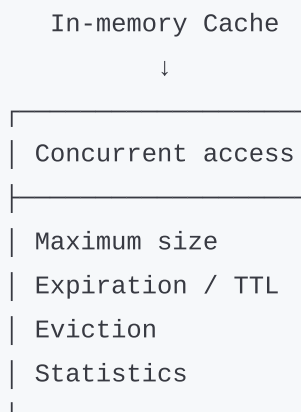
```
}  
}
```

`ConcurrentHashMap` handles concurrent access to individual map operations.

If I need LRU + TTL + maximum size

For a production application, I'd prefer a dedicated caching library rather than manually synchronizing a `HashMap`.

Conceptually:



Important design considerations

1. Thread safety
2. Maximum cache size
3. Eviction policy → LRU / size-based
4. TTL / expiration
5. Cache miss handling
6. Concurrent loading / stampede protection
7. Memory limits
8. Metrics and monitoring

★ 5-year interview answer

For a simple thread-safe key-value cache, I'd use `ConcurrentHashMap`. If the cache needs production-grade features such as maximum size, TTL, eviction, statistics, and concurrent loading, I'd use a caching library like Caffeine rather than implementing those mechanisms manually. If I specifically need an LRU implementation for an interview or simple use case, `LinkedHashMap` with access-order can be used, but I'd add appropriate synchronization because `LinkedHashMap` itself is not thread-safe.